Fast Discovery of Conditional Functional Dependencies via Transformer-Guided Relation Partitioning

Ruochun Jin, Shenglin Chen†, Xi Wang, Siyi Yang, Ting Wang
College of Computer Science and Technology, National University of Defense Technology, Changsha, China
jinrc@nudt.edu.cn,13272068106@163.com,18342211026@163.com,yangsi_@nudt.edu.cn,tingwang@nudt.edu.cn

ABSTRACT

Discovering conditional functional dependencies (CFDs) in large relations has been a long-standing challenge due to prohibitive computational costs. We propose SCFDM, a Transformer-guided framework that accelerates the discovery by modeling CFDs as latent conditional distributions. By projecting relational data into a high-dimensional space via a Transformer-based model, SCFDM leverages self-attention saliency to identify correlated attributes by capturing their interaction strengths. This neural-driven insight allows SCFDM to divide the exponentially large search space into independent smaller ones. To ensure discovery quality, we propose a diversity-aware sampling strategy with a theoretical lower bound for discovery accuracy, which reduces input size with performance guarantee of CFD discovery. This Transformer-guided division of the input relation enables data parallelism for any existing CFD discovery algorithm. Extensive experiments on seven real-world and synthetic datasets demonstrate that SCFDM achieves an average speedup of 107.78× (up to 171.14×) compared with baselines while maintaining high discovery accuracy of over 90%.

1 INTRODUCTION

Data dependencies, such as conditional functional dependencies (CFDs) [24], matching dependencies (MDs) [22] and functional dependencies (FDs) [15], are crucial in data management tasks, including query optimization [43], data cleaning [23], and data repair [23]. However, dependency discovery remains challenging, especially for those that contain patterns such as CFDs [27]. It is costly to discover CFDs, as it involves both identifying FDs embedded in CFDs and mining associated patterns [34], which takes $O(m \times d^m \times n)$ [62], where m, n and d are the number of attributes, tuples, and average domain size of attributes, respectively. To accelerate CFD discovery, researchers have proposed various algorithms [12, 14, 19, 20, 26, 33, 34, 39, 42, 47, 48, 62, 73, 77] with optimization strategies as follows. (1) Pruning strategies eliminate invalid candidates early to reduce the search space [26, 47, 62, 77]. (2) Tuple sampling selects subsets from the original relation to handle large datasets [6, 8, 27, 44, 50]. (3) Parallel mining employs multi-processor processing to accelerate the discovery [27, 28, 59].

However, previous methods still face the following challenges in practice. First, most existing methods discover directly on the input relation by level-wise search, where the exploration space grows exponentially with the number of attributes [26, 56]. This incurs unbearable enumeration and high memory cost when the input scales in columns. Although the support-based pruning [26–28] helps to alleviate this problem, the discovery can hardly terminate in practice when a low support threshold is set. Second, existing parallel paradigms suffer from excessive synchronization overhead as they manage interdependent search spaces [27]. As the pruning and validation of rules often depend on the state of other processors, the parallel threads frequently become idle while waiting for synchronization, resulting in high communication cost to maintain consistency across multiple cores [46]. Such dependencies among the cores makes it difficult to achieve linear speedup and optimal load balancing [27]. Third, although most existing sampling methods (e.g., single-round or reservoir sampling) operate within accuracy bounds [8, 11, 27, 76], they treat each tuple equally when sampling and ignore the semantic correlations among them. Thus, these sampling methods cannot skip unrelated rows in which CFDs are unlikely to be discovered and may miss relatively rare tuples where low-support CFDs are embedded, which cause redundant search and fail to discover valid CFDs of low support.

These challenges lead to the following questions: (1) Can we capture correlations among attributes before discovery to prune the search space? (2) How can we sample tuples such that the majority of CFDs, including low-support ones, can be discovered from the sampled data? (3) Can we partition the input relation into independent sub-tables based on attribute correlations to accelerate CFD mining via data parallelism? (4) Given noisy real-world datasets, can we discover CFDs that are valid for data management tasks?

Contributions & organization. In view of these challenges, we propose SCFDM, a **S**ub-table-based **C**onditional **F**unctional **D**ependency data-parallel **M**ining framework that accelerates CFD discovery via data parallelism where the input relation is divided into independent sub-tables. Specifically, each sub-table is constructed by grouping correlated attribute sets identified by a Transformer-based model and sampling similar tuples via a representative sampling strategy. SCFDM allows existing CFD discovery algorithms to be executed on these sub-tables in parallel, which significantly accelerates the mining process while maintaining accuracy. SCFDM has the following features.

(1) Vector Representation for CFD *Discovery (Section 5).* We develop a two-stage embedding pipeline that transforms raw relations into vector representations for correlation learning. We first employ one-hot encoding to convert discrete categorical values into a sparse zero-one matrix M, which is subsequently utilized to identify similar tuples that support CFDs. Vectors in M are then projected into a dense latent space to construct the input tensor $\mathcal{T}$, enabling the Transformer-based model to capture attribute-level correlations and identify the correlated attribute sets required for mining CFDs.

(2) Probabilistic Modeling of CFDs *(Section 6).* We define CFDs from a statistical perspective, which formulates attribute depen-

†The corresponding author.

dencies as conditional probability distributions. By approximating these distributions, the Transformer-based model can effectively capture and learn the correlations of attributes in CFDs.

(3) Attention-driven Correlated Attribute Set Extraction (Section 6). We propose AttrFinder, a Transformer-guided module that extracts correlated attribute sets by analyzing the Transformer's global attention distributions. Instead of enumerating in the exponentially large search space, SCFDM identifies the correlations among attributes by analyzing the attention maps of the Transformer. This approach extracts highly correlated attribute sets from the learned attention weights, which are then used to partition the input relation into independent sub-tables. These sub-tables enables efficient data parallelism for existing CFD discovery algorithms.

(4) Diversity-Aware Representative Tuple Sampling (Section 7). To reduce computational overhead while providing a discovery recall guarantee, SCFDM establishes a theoretical sampling lower bound. By incorporating binomial variance into the probabilistic analysis of CFD support, SCFDM derives a tighter sample budget compared to standard Chernoff bounds. To populate this budget, we employ diversity-first selection guided by locality sensitive hashing (LSH), which partitions tuples into similarity-aware bucket. This ensures uniform sampling coverage across all data buckets, which prevents low-support CFDs from being obscured by frequent patterns while maintaining the diversity of CFDs within the sampling bound.

(5) Parallel Discovery of CFDs *(Section 8).* SCFDM discovers CFDs through data parallelism guided by correlated attribute sets and representative tuples, where sampled representative tuples are projected onto correlated attribute sets to partition the original relation into smaller independent sub-tables that serve as parallel inputs.

(6) Experimental Study (Section 9). Using seven real-world and synthetic datasets, we validate the following results: (a) SCFDM speeds up classical CFD mining algorithms by 107.78 times on average, while maintaining over 90% accuracy. (b) The runtime of SCFDM is robust to its parameters. For example, when the normalized support threshold decreases from 0.5 to 0.1 on the Census dataset, its runtime increases by only 2.80 times, compared to a 5.30 times increase in the runtime of CFDMiner [26]. (c) SCFDM scales effectively with more processing cores. For instance, it achieves a 2.48× speedup on the RT dataset when the number of cores increases from 28 to 112.

Novelty. The novelty of SCFDM includes: (1) a probabilistic perspective that formulates CFDs as latent conditional distributions that can be captured by a Transformer-based model; (2) an attention-driven module to extract correlated attribute sets and divide the exponentially large search space into smaller ones; (3) a diversity-aware sampling strategy with a lower bound that preserves discovery recall while reducing the input size; and (4) a data parallelism framework that constructs sub-tables by grouping correlated attribute sets and representative sampling tuples, which accelerates existing CFD discovery algorithms via parallel mining.

2 RELATED WORK

CFD discovery. Researchers have proposed various CFD discovery algorithms as follows: (1) Level-wise search with pruning is widely employed for discovering CFDs, including CTANE [26], constant-pattern-generation-based [34, 66], attribute-grid-based [12], FACD [47] and CFUN [19]. (2) Depth-first search. FastCFD [25] uses a depth-first strategy with closed-itemset pruning, while BFCFD [48] employs sampling for single-pass discovery. (3) Hybrid approaches. As shown in [62], these methods first mine patterns from itemsets and then discover FDs in the corresponding subsets, or first discover FDs and then mine patterns applicable to those FDs. (4) Others. A greedy approximation algorithm computes near-optimal tables of CFDs from embedded FDs [34]. CFDMiner employs generators and closures for efficient constant CFD discovery [26]. GCFD discovers CFDs by detecting data errors guided by user-annotated values [42]. XPlode identifies the top-ranking CFDs for data repair tasks using an on-demand approach [61].

Different from previous methods, SCFDM formulates the attribute dependencies in CFDs as conditional distributions that can be learned by a Transformer-based model. By analyzing the self-attention saliency maps of the Transformer, SCFDM extracts correlated attribute sets for parallel discovery. This Transformer-guided approach enables fast discovery of CFDs via data parallelism that can be easily integrated with any existing discovery method.

Tuple sampling. Tuple sampling has been applied to dependency discovery [2, 7, 26, 48, 57, 58, 67, 68], including random sampling [53], stratified sampling [30], reservoir sampling [72], and focused sampling [5]. (1) Random sampling selects tuples with equal probability, where Chernoff [76] and Chebyshev [50] bounds are employed to calculate sample size based on error and confidence thresholds. (2) Stratified sampling divides data into multiple sets before sampling, which reduces the sample size based on data distribution and transactions [49, 74], where the main challenge is how to partition the strata [54]. (3) Reservoir-sampling-based CFD discovery iteratively samples data and refines the discovered rules. During this process, each tuple is evaluated to determine whether its attribute values satisfy the rules [48]. (4) Focused sampling targets at tuple pairs specified by consistency sets, evidence sets [8, 44], or errors in user-annotated attributes [42].

SCFDM differs from previous methods in that it not only samples tuples but also divides the input relation by selected columns. Specifically, based on the attention mechanism of Transformers, it reduces the search space by projecting the input onto correlated attributes where candidate CFDs are embedded. Moreover, theoretical analysis provides performance guarantee of support and confidence bounds for the sampling strategy in SCFDM.

Parallel dependency discovery. Various parallel optimizations for dependency discovery have been proposed. ParaCoDe uses a depth-first strategy with a single-node shared-everything architecture [63] to mine CFDs in parallel within a partitioned search tree, which may however suffer from node overloads [32]. Saxena et al. introduce six primitives to parallelize dependency discovery, which focus on workload allocation [40]. However, these primitives do not support CFD patterns [27]. Naumann et al. parallelize approximate dependency mining by traversing multiple search spaces [44]. Han et al. propose parallel rule discovery algorithms that ensure scalability with consideration of computational overhead [27, 28]. However, as data scales, these algorithms may encounter potential bottlenecks in communication overhead and adaptive task allocation, partly due to the static coupling between search spaces and

Table 1: Example Relation of Adult Information

ID	Name	Age	Workclass	Education	Edu-num	Relationship	Gender	Occupation	Cc	Ac	City	Proto	Service
t_1	Andy	23	Self-emp	Bachelors	13	Not-in-family	Male	Exec-managerial	05	18	LD	TCP	MQTT
t_2	Edward	34	State-gov	Bachelors	13	Husband	Male	Adm-clerical	01	33	NY	UDP	DNS
t_3	Tom	25	Private	HS-grad	9	Husband	Male	Handlers-cleaners	08	22	MS	TCP	MQTT
t_4	Angelia	33	Private	Masters	14	Wife	Female	Exec-managerial	05	18	LD	UDP	DNS
t_5	Justin	36	Self-emp	Masters	14	Husband	Male	Exec-managerial	08	22	MS	TCP	MQTT
t_6	Alex	42	State-gov	Bachelors	13	Not-in-family	Male	Prof-specialty	01	33	NY	UDP	DNS
t_7	Candice	29	State-gov	Bachelors	13	Wife	Female	Prof-specialty	01	33	NY	TCP	MQTT

resources, which hinders dynamic load balancing.

Different from previous parallel solutions, SCFDM offers a data-parallel framework that can be integrated with any existing CFD mining algorithm, which inherits well-designed optimizations (e.g. pruning strategies and auxiliary data structures in [26]) and the advanced serial performance of previous methods. Moreover, since the CFD discovery on each sub-table is independent, SCFDM enables flexible load balancing and reduces the communication overhead.

3 PRELIMINARIES

Consider a relational schema $\mathcal{R}$ defined over a set of attributes $attr(\mathcal{R})$, where each attribute $A \in attr(\mathcal{R})$ has a domain $dom(A)$. For each tuple t in relation r of $\mathcal{R}$, $t[A] \in dom(A)$, where $t[A]$ is the value of t for attribute A. Next, we review the basics of CFDs.

Definition 3.1: *Given a relation r over $\mathcal{R}$, a set of attributes $attr(\mathcal{R})$, and a pattern tuple t_p, a conditional functional dependency φ is defined as:*

$$\varphi : (\boldsymbol{X} \rightarrow Y, t_p),$$

where $\boldsymbol{X}$ is a set of attributes in $attr(\mathcal{R})$, Y is an attribute in $attr(\mathcal{R})$, and $\boldsymbol{X} \rightarrow Y$ denotes the functional dependency embedded in φ. The pattern tuple t_p is defined for $\boldsymbol{X} \rightarrow Y$, where for each attribute $A \in \boldsymbol{X} \cup Y$, the value of $t_p[A]$ is a constant "a" from $dom(A)$ or an unnamed variable "_" that can take any value from $dom(A)$. φ is a variable CFD *if t_p contains any "_", and a constant* CFD *otherwise.*

Semantics of CFDs. Following [26], an order $\leq$ on constants and the unknown variable "_" is defined as follows: $\eta_1 \leq \eta_2$ if $\eta_1 = \eta_2$, or η_1 is a constant and η_2 is "_". The order $\leq$ naturally extends to tuples, e.g., $(\text{Male}, \text{UDP} \parallel \_) \leq (\_, \text{UDP} \parallel \_)$. $t_{p1}[\mathbf{X}] \ll t_{p2}[\mathbf{X}]$ if $t_{p1}[\mathbf{X}] \leq t_{p2}[\mathbf{X}]$ but $t_{p2}[\mathbf{X}] \not\leq t_{p1}[\mathbf{X}]$, i.e., t_{p2} is more general than t_{p1}. For example, $(\text{Male}, \text{UDP} \parallel \_) \ll (\_, \text{UDP} \parallel \_)$. Given a CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ and an instance r of $\mathcal{R}$, we say that r satisfies φ, denoted by $r \models \varphi$, iff for each pair of tuples t_1, t_2 in r, if $t_1[\mathbf{X}] = t_2[\mathbf{X}] \leq t_p[\mathbf{X}]$, then $t_1[Y] = t_2[Y] \leq t_p[Y]$.

Example 1: *Table 1 consists of the following* CFDs*: φ_1 : ([Workclass] $\rightarrow$ Education, (State-gov || Bachelors)), φ_2 : ([Education] $\rightarrow$ Edu-num, (_ || _)) and φ_3 : ([Gender, Proto] $\rightarrow$ City, (_, UDP || _)). Table 1 does not satisfy φ : ([Cc] $\rightarrow$ Proto, (_ || _)), as t_1 and t_4 violate φ, where $t_1[Cc] = t_4[Cc] \leq$ "_", but $t_1[Proto] \neq t_4[Proto]$.*

Support. As defined in [26], the support of a CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ in relation r, denoted by $\text{supp}(\varphi, r)$, is the size of set $r_\varphi = \{t | t \in r, t[\mathbf{X} \cup Y] \leq t_p[\mathbf{X} \cup Y]\}$ such that for any $t_1, t_2 \in r_\varphi$, if $t_1[\mathbf{X}] = t_2[\mathbf{X}]$, then (1) $t_1[Y] = t_2[Y]$, and (2) $t_1[Y] \leq t_p[Y]$. Here, (1) enforces the semantics of the embedded FD $\mathbf{X} \rightarrow Y$, and (2) enforces the domain constraints of pattern t_p applied to $\mathbf{X} \rightarrow Y$. φ is *σ-frequent* if $\text{supp}(\varphi, r) \geq \sigma$, where σ is the support threshold.

Confidence. Following the definitions in [42], the confidence of a CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ in r, denoted by $\text{conf}(\varphi, r)$, is defined by $\text{conf}(\varphi, r) = \frac{\text{supp}(\varphi, r)}{\text{supp}(\mathbf{X}, r)}$, where $\text{supp}(\varphi, r)$ is the support of φ over r and $\text{supp}(\mathbf{X}, r)$ is the size of set $r_{\mathbf{X}} = \{t | t \in r, t[\mathbf{X}] \leq t_p[\mathbf{X}]\}$. A CFD φ is *δ-confident* if $\text{conf}(\varphi, r) \geq \delta$, where δ is a given threshold.

Definition 3.2: *Following Definition 3.1, given the support and confidence thresholds σ and δ, and a relation r, a* CFD *φ over r that satisfies σ and δ is defined as:*

$$\varphi : (\boldsymbol{X} \rightarrow Y, t_p), s.t.\ \text{supp}(\varphi, r) \geq \sigma \wedge \text{conf}(\varphi, r) \geq \delta.$$

The confidence threshold helps to mine CFDs in real-life data with noise, where CFDs that violate a few noisy tuples can still be discovered. Meanwhile, the support threshold filters out trivial or complex CFDs that overfit noisy data [75], since support is anti-monotonic and efficiently prunes invalid CFDs during the rule discovery process [26].

Example 2: *Continue with Example 1, given a* CFD *φ_4 : ([Gender] $\rightarrow$ Relationship, (Male || Husband)), we have $\text{supp}(\varphi_4, r) = 3$ (supported by $r_\varphi = \{t_2, t_3, t_5\}$), $\text{supp}(\varphi_1, r) = 3$ (supported by $r_\varphi = \{t_2, t_6, t_7\}$) and $\text{supp}(\varphi_2, r) = 7$, $\text{conf}(\varphi_3, r) = 1$ (not violated by $r_{\mathbf{X}} = \{t_2, t_4, t_6\}$), and $\text{conf}(\varphi_4, r) = \frac{3}{5}$ (violated by $r_{\mathbf{X}} \setminus r_\varphi = \{t_1, t_6\}$).*

Minimality of CFDs. A CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ over r is *nontrivial* if $Y \notin \mathbf{X}$. Under the support (σ) and confidence (δ) thresholds, (1) a constant [26] CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ is *left-reduced* over r if for any subset $\mathbf{X}' \subset \mathbf{X}$, $r \not\models (\mathbf{X}' \rightarrow Y, t_p)$; (2) a variable [26] CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ is *left-reduced* over r if $r \not\models (\mathbf{X}' \rightarrow Y, t_p)$ for any subset $\mathbf{X}' \subset X$, and $r \not\models (\mathbf{X} \rightarrow Y, t_p')$ for any t_p' with $t_p \ll t_p'$. A *minimal* CFD φ over r is *nontrivial* and *left-reduced*.

Cover of CFDs. Consider a set Σ of *minimal* CFDs on r [27]. We say that Σ *entails* a CFD φ, denoted by $\Sigma \vDash \varphi$, if $r \vDash \Sigma$ deduces $r \vDash \varphi$. Σ is *equivalent* to another set Σ', denoted by $\Sigma \equiv \Sigma'$, if $\Sigma \vDash \varphi'$ for any $\varphi' \in \Sigma'$ and vice versa. Σ is *minimal* if for each CFD $\varphi \in \Sigma$, Σ is not *equivalent* to $\Sigma \setminus \varphi$, i.e., each CFD φ in Σ is *non-redundant*. A cover Σ_c of Σ on r is a subset of Σ such that (1) $\Sigma_c \equiv \Sigma$, (2) all rules φ in Σ_c are *minimal*, and (3) Σ_c is minimal, i.e., Σ_c includes no *redundant* rules.

Problem statement. Given a noisy relation r of schema $\mathcal{R}$, support and confidence thresholds σ and δ, the CFD discovery problem is to identify a minimal cover Σ consisting of all minimal CFDs over r that satisfy the thresholds σ and δ.

4 SOLUTION OVERVIEW

As shown in Figure 1, SCFDM takes a relation r as input and outputs a minimal cover of CFDs. Each module operates as follows.

Relational Encoding and Attribute Embedding. This module

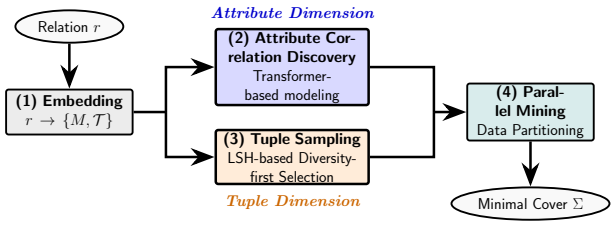


Figure 1: Overview of SCFDM

converts the relation r into vector representations via a two-stage process (Section 5). It first maps discrete values to a zero-one matrix M, which is utilized to construct the high-dimensional input tensor $\mathcal{T}$ for the Transformer-based model and facilitate representative tuple sampling. Then, these vectors in M are projected into dense latent states $\mathbf{E}_i$ in $\mathcal{T}$, which enables the self-attention mechanism to measure attribute-wise similarities in relation r by dot-product of vectors. Furthermore, the row-wise similarity within the matrix M provides a quantitative metric for representative tuple sampling, as it is more possible to discover CFDs from similar tuples.

Correlated attribute sets discovery. This module identifies attribute dependencies by modeling a CFD as a conditional probability distribution (Section 6). It employs a Transformer encoder to learn joint distributions through masked value reconstruction [17, 37], capturing attribute correlations via the self-attention mechanism. Based on the global attention maps in Transformer [71], it takes $O(m^2)$ for the module to identify candidate antecedent sets $\mathbf{X}$ for each consequent Y. These identified correlated attribute sets facilitate data parallelism by guiding the partition of the relation into independent sub-tables for accelerated CFD discovery.

Representative tuple sampling. This module extracts representative tuples from r with a discovery recall guarantee (Section 7). It first derives a sampling lower bound N, which reduces sample size by incorporating binomial variance. To preserve discovery recall, this module employs similarity-aware stratified sampling via locality sensitive hashing (LSH), partitioning tuples into similarity buckets in linear time. By adopting a diversity-first selection logic, it ensures uniform coverage across all data buckets and prevents low-support CFDs from being obscured by dominant patterns, thereby maintaining the diversity of CFDs within the sampling budget N.

Parallel discovery of CFDs**.** SCFDM projects the sampled representative tuples to multiple correlated attribute sets and produces sub-table inputs for CFD discovery. Then, SCFDM employs existing discovery algorithms to find CFDs from these independent sub-tables in parallel. That is, SCFDM can parallelize any CFD discovery algorithm via data parallelism (Section 8).

Specifically, Figure 2 shows the pipeline of the Transformer-based model in SCFDM that takes the relation r as input and yields the correlated attribute sets $Corr$ of r, which includes vectorization of input relation (Section 5), correlation learning via Transformer (Section 6.2), and attention-based attribute set discovery (Section 6.3). The process begins with vectorization, where a relation r over $attr(\mathcal{R})$ is transformed into a high-dimensional tensor $\mathcal{T} \in \mathbb{R}^{n \times m \times d}$ by firstly applying one-hot encoding to convert all tuples in r into a zero-one matrix $M \in \{0,1\}^{n \times \eta}$ followed by an attribute embedding layer that projects M into tensor $\mathcal{T}$. This tensor $\mathcal{T}$ serves as the input for the Transformer-based learning module,

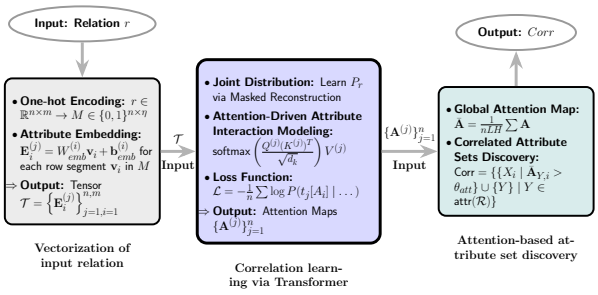


Figure 2: Pipelines of Transformer-based model

which employs a Transformer encoder to perform masked attribute modeling. By minimizing the reconstruction loss $\mathcal{L}$ across all tuples, the model learns the conditional probability distribution $P_r(A_i \mid attr(\mathcal{R}) \setminus \{A_i\})$ and generates attention maps $\{\mathbf{A}^{(j)}\}$ that record the correlations among attributes. These attention maps are then fed into the attention-based attribute set discovery module, which performs global attention analysis to construct a global attention map $\bar{\mathbf{A}}$. We then iteratively leverage $\bar{\mathbf{A}}$ to identify, for each attribute Y, its correlated antecedents $\mathbf{X}$ that exceed a significance threshold, thereby constructing the set of correlated attribute sets $Corr$ in $O(m^2)$ time.

5 VECTORIZATION OF INPUT RELATION

This section details the two-stage vectorization of input relation r. We first employ one-hot encoding (Section 5.1) to convert discrete categorical values into a zero-one matrix M. To facilitate the learning of attribute correlations in CFDs, we further design an attribute embedding layer (Section 5.2) that projects zero-one vectors in M into tensor $\mathcal{T}$. This vectorization enables the Transformer to learn and quantify the attribute correlations by computing dot-product similarities between vectors in $\mathcal{T}$.

5.1 One-hot Encoding of Tuples

Given a relation r over the attribute set $attr(\mathcal{R}) = \{A_1, \ldots, A_m\}$, let $dom(A_i)$ denote the domain of each attribute A_i with $c_i = |dom(A_i)|$. Each value $a \in dom(A_i)$ is encoded as a zero-one vector $\mathbf{v}_i \in \{0,1\}^{c_i}$, where only the j-th element x_j is set as 1 if a is the j-th value (alphabetically ordered) in $dom(A_i)$. The representation of each tuple $t_j \in r$ is the concatenation of zero-one vectors $\mathbf{m}_j = [\mathbf{v}_1^\top, \mathbf{v}_2^\top, \ldots, \mathbf{v}_m^\top]$ that corresponds to its m attribute values (ordered alphabetically by attribute names in $attr(\mathcal{R})$), where $\mathbf{m}_j \in \{0,1\}^{1 \times \eta}$ and $\eta = \sum_{i=1}^{m} c_i$. Thus, the relation r is transformed into a zero-one valued matrix $M \in \{0,1\}^{n \times \eta}$ by encoding each tuple as a zero-one vector.

5.2 Embedding of Attributes and Relations

Given the zero-one encoding matrix M introduced in Section 5.1 as input, in order to enable a Transformer [70] to capture correlations among attributes, we design an attribute embedding layer as shown in Figure 2. Specifically, for a row $\mathbf{m}_j = [\mathbf{v}_1^\top, \mathbf{v}_2^\top, \ldots, \mathbf{v}_m^\top]$ in M, each segment $\mathbf{v}_i \in \{0,1\}^{c_i}$ $(1 \le i \le m)$ is projected to a d-dimensional vector via an embedding layer (similar to [38]):

$$\mathbf{E}_i^{(j)} = W_{emb}^{(i)} \mathbf{v}_i + \mathbf{b}_{emb}^{(i)},$$

where $\mathbf{E}_i^{(j)} \in \mathbb{R}^d$ represents the embedding of attribute A_i in tuple t_j. Thus, a tuple $t_j \in r$ is transformed into an matrix $\mathbf{H}_{in}^{(j)} =$

$[\mathbf{E}_1^{(j)}, \ldots, \mathbf{E}_m^{(j)}]^\top \in \mathbb{R}^{m \times d}$ and we denote the trainable weights of the attribute embedding layer as $\Theta_{emb} = \{W_{emb}^{(i)}, \mathbf{b}_{emb}^{(i)}\}_{i=1}^{m}$ (weight details refer to Section 6.2 and Appendix A in [1]). These parameters Θ_{emb} are randomly initialized and jointly optimized with the downstream Transformer (Section 6). By embedding all n rows in M, the attribute embedding layer produces a tensor $\mathcal{T} \in \mathbb{R}^{n \times m \times d}$, which enables the Transformer to treat the relation as a sequence of m attribute embeddings for each of the n tuples. This vectorized sequence allows the Transformer to learn and capture the correlations among attributes via self-attention mechanism.

Remark. Modeling the input relation r as a sequence of dense embeddings $\mathbf{E}_i$ is pivotal for discovering attribute correlations. By projecting diverse attribute domains to a uniform d-dimensional space, the self-attention mechanism of the Transformer can compute dot-product similarities between attribute identities, which quantifies the correlation between a candidate antecedent set $\mathbf{X}$ and a consequent Y. Thus, attribute correlations within CFDs, which are hidden in the original relation, become measurable through the dot-products of the embeddings $\mathbf{E}_i$.

6 CORRELATED ATTRIBUTE DISCOVERY

In this section, we first formulate CFDs as conditional probability distributions (Section 6.1). Based on this formulation, we train a Transformer model using the embedding matrix $\mathcal{T}$ derived in Section 5.2 to learn the attribute correlations by modeling these conditional distributions (Section 6.2). Finally, we demonstrate how the attention maps within the trained Transformer can be utilized to capture and identify correlated attribute sets in CFDs (Section 6.3).

6.1 CFD as Conditional Probability

Inspired by the statistical analysis of FDs in [75], we first analyze CFDs from a statistical perspective. Following [31, 75], consider a tuple set $r_\varphi \subseteq r$ where $r_\varphi = \{t | t \in r, t[\mathbf{X} \cup Y] \leq t_p[\mathbf{X} \cup Y]\}$, and for any two tuples $t_i, t_j \in r_\varphi (i \neq j)$, if $t_1[\mathbf{X}] = t_2[\mathbf{X}]$, then $t_1[Y] = t_2[Y] \leq t_p[Y]$. A CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ is a statement over the attribute set $\mathbf{X} \subseteq attr(\mathcal{R})$ and an attribute $Y \in attr(\mathcal{R})$, where an assignment to $\mathbf{X}$ uniquely determines the value of Y on the subset r_φ above support and confidence thresholds. Thus, $\wedge_{A \in \mathbf{X}} t_i[A] = t_j[A]$ and $t_i[Y] = t_j[Y]$ can be viewed as two associated events, and a CFD can be written as the conditional probability of event $t_i[Y] = t_j[Y]$ given $\wedge_{A \in \mathbf{X}} t_i[A] = t_j[A]$ as the condition.

Definition 6.1: *Given a* CFD $\varphi : (\boldsymbol{X} \rightarrow Y, t_p)$*, a tuple set* $r_\varphi \subseteq r$ *that conforms with* φ*, the conditional probability representation of* φ *is* $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A \in \boldsymbol{X}} t_i[A] = t_j[A]) \geq \delta$.

Here, $P_r(t_i[Y] = t_j[Y] | \wedge_{A \in \mathbf{X}} t_i[A] = t_j[A]) \geq \delta$ is the conditional probability between $\mathbf{X}$ and Y restricted to r_φ. Consider the CFD φ in Definition 6.1, where $\mathbf{X} = \{A_1, A_2, \cdots, A_k\}$ is a multi-variable set and Y is a variable. The domain of $\mathbf{X}$, denoted as $dom(\mathbf{X})$, is determined by r_φ, i.e., $dom(\mathbf{X}) = \{t_i(A_1) \times t_i(A_2) \times \cdots \times t_i(A_k)\}$ for all $t_i \in r_\varphi$. When $\mathbf{X}$ is assigned by values in $dom(\mathbf{X})$, there exists a function of conditional association $Y = f(\mathbf{X})$ from $\mathbf{X}$ to Y such that if $y = f(\mathbf{x})$ then $\forall \mathbf{x} \in dom(\mathbf{X}) : P_r(Y = y | \mathbf{X} = \mathbf{x}) \geq \delta$. That is, when $\mathbf{X}$ takes values in its domain $dom(\mathbf{X})$, Y corresponds to a value $f(\mathbf{X})$ with at least δ probability, as constrained by $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A \in \mathbf{X}} t_i[A] = t_j[A]) \geq \delta$.

Algorithm 1 AttrFinder (Attention-based Discovery)

Input: Relation r (as Transaction Matrix $M \in \{0,1\}^{n \times \eta}$), mask probability p_{mask}, attention threshold θ_{att}, confidence τ_c, stability σ^2_{max}
Output: Correlated attribute sets $Corr$
$Corr \leftarrow \emptyset$
while not converged **do**
  $\mathcal{B} \leftarrow \text{SampleBatch}(M)$
  $\tilde{\mathcal{B}} \leftarrow \text{ApplyMask}(\mathcal{B}, p_{mask})$
  $\mathcal{T}_{\mathcal{B}} \leftarrow \text{Embedding}(\tilde{\mathcal{B}})$
  $\hat{Y} \leftarrow \text{TransformerEncoder}(\mathcal{T}_{\mathcal{B}})$
  $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} \text{CrossEntropy}(M_j, \hat{Y}_j)$
  $\text{UpdateWeights}(\mathcal{L})$
$\bar{\mathbf{A}} \leftarrow \frac{1}{n \cdot L \cdot H} \sum_{j=1}^{n} \sum_{l=1}^{L} \sum_{h=1}^{H} \mathbf{A}_{j,l,h}$
for each attribute $Y \in attr(\mathcal{R})$ **do**
  $\mathbf{Row}_Y \leftarrow \bar{\mathbf{A}}[Y, \cdot]$
  $\theta_{att} \leftarrow k/m$
  $\mathbf{X}_Y \leftarrow \{A_i \mid \bar{\mathbf{A}}[Y, i] \geq \theta_{att}, A_i \neq Y\}$
  if $\mathbf{X}_Y$ is not empty **then**
    $Corr_Y \leftarrow \mathbf{X}_Y \cup \{Y\}$
    $Corr \leftarrow Corr \cup \{Corr_Y\}$
return $Corr$

6.2 Correlation Learning via Transformer

In order to learn the conditional probability of CFDs, we formulate an end-to-end reconstruction task for the Transformer, where correlations among attributes are captured by learning the conditional probability $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A \in \mathbf{X}} t_i[A] = t_j[A]) \geq \delta$. Specifically, we treat each tuple vector in $\mathcal{T}$ as a sequence and train the Transformer model to recover masked attribute values, thereby completing the learning of conditional probabilities in CFDs driven by this representation reconstruction task. For each tuple t_j, every attribute $A_i \in \text{attr}(\mathcal{R})$ is iteratively selected and replaced with a special [MASK] token. The L-layer Transformer leverages the self-attention mechanism to integrate information from the $m-1$ observed attributes into the hidden state of the masked attribute. The output hidden representation in Transformer captures the attribute correlations collected via the self-attention mechanism: $\text{Attention}(Q^{(j)}, K^{(j)}, V^{(j)}) = \text{softmax}\left(\frac{Q^{(j)}(K^{(j)})^T}{\sqrt{d_k}}\right) V^{(j)}$, where $Q^{(j)}, K^{(j)}, V^{(j)}$ are linear projections of the attribute embeddings $\mathbf{H}_{in}^{(j)}$ in $\mathcal{T}$. Notably, $\mathbf{A}^{(j,l,h)} = \text{softmax}(Q^{(j,l,h)}(K^{(j,l,h)})^T/\sqrt{d_k})$ represents the self-attention map, which quantifies the dependencies among attributes. By stacking L layers (where $L = 6$ follows common practices for tabular data to balance efficiency and accuracy [35, 37]), the Transformer iteratively refines these maps to capture attribute correlations (e.g., X_1 and X_2 jointly determining Y). The attribute embedding layer and the Transformer model is jointly optimized by minimizing the average cross-entropy loss across all tuples:

$$\mathcal{L}(\Theta) = -\frac{1}{n} \sum_{j=1}^{n} \log P(t_j[A_i] \mid t_j[\text{attr}(\mathcal{R}) \setminus \{A_i\}]; \Theta),$$

which forces the Transformer to learn the conditional probability distribution P_r of relation r, where $\Theta = \{\Theta_{\text{emb}}, \Theta_{\text{trans}}\}$ is iteratively optimized including the attribute embedding parameters Θ_{emb} and

the Transformer parameters Θ_{trans}.

By minimizing $\mathcal{L}(\Theta)$, the Transformer maximizes the likelihood of predicting the true attribute values, thereby aligning its internal attention mechanism with the conditional distributions P_r of the CFDs. This optimization compels the Transformer to discover attribute correlations by focusing on attributes that provide the relatively stronger predictive signals for the masked targets. The parameters $\Theta = \{\Theta_{emb}, \Theta_{trans}\}$ are adjusted through backpropagation using the AdamW optimizer [51].

6.3 Attention-based Attribute Set Discovery

The extraction of correlated attribute sets in CFDs is achieved through the analysis of attention maps in the trained Transformer of Section 6.2. As shown in Algorithm 1, AttrFinder takes two steps to identify correlated attribute sets.

Step 1: Attention Map Extraction (Lines 4–11). Following the jointly optimization process described in Section 6.2 (Lines 4–10), this optimization process compels the attention maps $\{\mathbf{A}^{(j)}\}_{j\in\mathcal{B}}$ to quantify the attribute correlations required to model the conditional probabilities of CFDs. Then, AttrFinder computes the global attention map $\bar{\mathbf{A}} \in \mathbb{R}^{m\times m}$ by averaging the attention maps across all n tuples, L layers, and H heads (Line 11):

$$\bar{\mathbf{A}} = \frac{1}{n \cdot L \cdot H} \sum_{j=1}^{n} \sum_{l=1}^{L} \sum_{h=1}^{H} \mathbf{A}^{(j,l,h)},$$

where each entry $\bar{\mathbf{A}}_{i,k}$ quantifies the correlation between attribute A_i and A_k. A higher value of $\bar{\mathbf{A}}_{i,k}$ indicates a stronger conditional probability P_r in CFDs.

Step 2: Correlated Attribute Sets Identification (Lines 12–19). AttrFinder utilizes the global attention map $\bar{\mathbf{A}}$ in Step 1 to identify correlated attribute sets of CFDs. By iteratively treating each attribute $A_i \in \text{attr}(\mathcal{R})$ as a consequent Y (Line 12), AttrFinder retrieves the row corresponding to attribute Y from the global attention map $\bar{\mathbf{A}}$ (Line 13), which captures the cumulative influence of all other attributes on Y. We then determine the candidate antecedent set $\mathbf{X}$ by selecting attributes whose value of $\bar{\mathbf{A}}_{i,k}$ exceed a threshold θ_{att}. Following common practice in attention analysis [16, 71], we set $\theta_{\text{att}} = \frac{k}{m}$ (Line 14), where $k > 1$ is a scaling factor, filtering out attributes with negligible predictive contribution relative to a uniform distribution of $1/m$. For each consequent attribute Y, the resulting correlated attribute set is $Corr_Y = \{X_i \mid \bar{\mathbf{A}}_{Y,i} > \theta_{att}\} \cup \{Y\}$ (Lines 15–17). The collection of correlated attribute sets for relation r is obtained by iterating over all attributes in attr($\mathcal{R}$): $Corr = \{Corr_Y \mid Y \in attr(\mathcal{R})\}$ (Line 18).

Example 3: *We illustrate the identification of the correlated attribute sets using the relation in Table 1. Consider the attribute City as the consequent Y.* AttrFinder *first retrieves the corresponding row from the global attention map $\bar{\mathbf{A}}$, which collects the attribute correlations across all n tuples.* AttrFinder *observes the attention weights $\bar{\mathbf{A}}[City, Gender] = 0.35$ and $\bar{\mathbf{A}}[City, Proto] = 0.42$. Given a scaling factor $k = 3$ and $m = 14$ attributes, these scores significantly exceed the threshold $\theta_{att} = 3/14 \approx 0.21$. Thus, attributes $\{Gender, Proto\}$ are selected as the candidate antecedent set $\mathbf{X}$, effectively pruning the attribute enumeration space into a localized correlated attribute set $Corr_{City} = \{Gender, Proto, City\}$.*

Complexity. The complexity of AttrFinder is $O(n \cdot m^2)$, where n is the number of tuples and m is the number of attributes; other factors such as the number of epochs, Transformer layers, and embedding dimension are treated as constants (see [1] for a detailed analysis of AttrFinder's components).

7 SAMPLING WITH ACCURACY BOUNDS

Mining CFDs on massive datasets remains costly as the number of tuples grows. As improper sampling incurs loss of rules [27], in order to reduce input size while preserving discovery quality, we propose RepSampler, a sampling algorithm with accuracy guarantees that accelerates CFD discovery. We first derive a sampling lower bound for CFD discovery (Section 7.1), then select representative tuples from stratified partitions to ensure that the sampled tuples provides the diversity of CFDs (Section 7.2).

7.1 Lower Bound for Sampling

To mitigate the potential loss of CFDs when sampling tuples, we investigate the lower bound of the sample size with accuracy guarantees. Let Σ be the set of minimal CFDs holding on the original relation r, and Σ_s be the set discovered from the sampled relation r_s. We quantify the quality of CFD discovery by precision and recall, where $\text{precision}(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma_s|}$ reports the proportion of CFDs discovered from the sample r_s that are minimal and hold on the original relation r, and $\text{recall}(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma|}$ measures the fraction of CFDs in the original relation r that are identified from the sample r_s. Intuitively, $\text{precision}(\Sigma, \Sigma_s)$ quantifies the percentage of spurious rules that hold on the sample r_s and are invalid on the original relation r. While $\text{recall}(\Sigma, \Sigma_s)$ quantifies the loss of CFDs incurred by the sampling process.

Given a CFD φ, and N independently and identically distributed samples $t_1, t_2, \ldots, t_N$ randomly drawn from the relation r, we define $X_i (1 \le i \le N)$ as an indicator variable where $X_i = 1$ if t_i conforms with φ and $X_i = 0$ otherwise. The sum $X = \sum_{i=1}^{N} X_i$ follows a binomial distribution $X \sim B(N, \hat{\sigma})$, with $E[X] = N\hat{\sigma}$ and $Var(X) = N\hat{\sigma}(1-\hat{\sigma})$, where $\hat{\sigma} = \sigma/n$ is the normalized support. To derive a sampling lower bound N under recall guarantee β and error bound ϵ, we consider the following concentration inequalities. Notably, since X is a sum of Bernoulli trials, we can utilize the variance-sensitive form of concentration bounds (Bernstein-type) to obtain a tighter N compared to the standard Hoeffding's inequality [36]. First, we apply the variance-augmented Hoeffding-Bernstein inequality [9, 36]: $P_r(|X - E[X]| \ge N\epsilon) \le 2\exp\left(\frac{-(N\epsilon)^2}{2Var(X)+\frac{2}{3}N\epsilon}\right) \approx 2\exp\left(\frac{-2(N\epsilon)^2}{Var(X)}\right) \le 1 - \delta\beta$. The use of $Var(X)$ is justified since it accounts for the distribution's sparsity; when $\hat{\sigma}$ is small (typical for CFD discovery), $Var(X) \ll N$, yielding a tighter bound than the non-parametric version. From this inequality, we can derive the variance-augmented lower bound: $N \ge \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{1-\delta\beta}{2})}{-2\epsilon^2}$.

We next compare the proposed variance-augmented lower bound with commonly used bounds derived from other sampling inequalities [4, 13, 18]. Specifically, using Chebyshev's inequality $P_r(|X - E[X]| \ge N\epsilon) \le \frac{Var(X)}{(N\epsilon)^2} \le 1 - \delta\beta$ [18] , we obtain Cheby-

shev's lower bound $N \geq \frac{\hat{\sigma}(1-\hat{\sigma})}{\epsilon^2(1-\delta\beta)}$. According to Bennett's inequality $P_r(|X - N\hat{\sigma}| \geq N\epsilon) \leq 2\exp\left(-N\hat{\sigma}\Theta\left(\frac{\epsilon}{\hat{\sigma}}\right)\right) \leq 1 - \delta\beta$ [4], we can derive Bennett's lower bound $N \geq \frac{-\ln(\frac{1-\delta\beta}{2})}{(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}})-\epsilon}$. Finally, a hybrid Chernoff bound was introduced in [13], given by $N \geq \frac{3\tau}{\epsilon^2}\ln\left(\frac{2}{1-\delta\beta}\right)$.

Theorem 1: *For $X \sim B(N, \hat{\sigma})$ where $0 \leq \hat{\sigma} \leq 1$, the variance-augmented lower bound is tighter than both Chebyshev's and Bennett's lower bounds.*

Proof. The ratio r_c of the variance-augmented bound to Chebyshev's bound is: $r_c = \frac{\frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2}}{\frac{\hat{\sigma}(1-\hat{\sigma})}{\epsilon^2(1-\delta\beta)}} = \frac{(1-\delta\beta)\ln(\frac{2}{1-\delta\beta})}{2}$, Since $0 < \delta\beta \leq 1$, the function $f(u) = u\ln(2/u)$ (where $u = 1-\delta\beta$) reaches its maximum near $u = 0.73$ with $f(u) \approx 0.74 < 2$. Thus, $0 < r_c < 1$, proving it is tighter than Chebyshev's. Similarly, for the ratio r_b to Bennett's bound, we leverage the inequality $\ln(1+x) > \frac{2x}{2+x}$ and the Taylor expansion $(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}}) - \epsilon \approx \frac{\epsilon^2}{2\hat{\sigma}}$ for small ϵ: $r_b = \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2} \cdot \frac{(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}})-\epsilon}{\ln(\frac{2}{1-\delta\beta})}$ and $r_b \approx \frac{\hat{\sigma}(1-\hat{\sigma})}{2\epsilon^2} \cdot \frac{\epsilon^2}{2\hat{\sigma}} = \frac{1-\hat{\sigma}}{4} < 1$.

Theorem 2: *Given the normalized support threshold $\hat{\sigma}$, additive Chernoff error bound ϵ, error ratio τ, confidence δ, and recall β, when $X \sim B(N, \hat{\sigma})$ and $\hat{\sigma}(1-\hat{\sigma}) \leq 6\tau$, the sampling lower bound N is determined by the tighter of the variance-augmented Hoeffding bound and the hybrid Chernoff bound [13]:*

$$N = \begin{cases} \frac{\hat{\sigma}(1-\hat{\sigma})\ln\left(\frac{2}{1-\delta\beta}\right)}{2\epsilon^2}, & \hat{\sigma} \in [0, \hat{\sigma}_1] \cup (\hat{\sigma}_2, 1] \text{ when } \tau \leq \frac{1}{24} \\ \frac{3\tau}{\epsilon^2}\ln\left(\frac{2}{1-\delta\beta}\right), & \hat{\sigma} \in (\hat{\sigma}_1, \hat{\sigma}_2] \text{ when } \tau \leq \frac{1}{24} \\ \frac{\hat{\sigma}(1-\hat{\sigma})\ln\left(\frac{2}{1-\delta\beta}\right)}{2\epsilon^2}, & \hat{\sigma} \in [0, 1] \text{ when } \tau > \frac{1}{24} \end{cases}$$

where $\hat{\sigma}_1, \hat{\sigma}_2 = \frac{1 \mp \sqrt{1-24\tau}}{2}$.

Proof. The proof consists of the following three parts.

(1) Size of samples. The ratio of the variance-augmented bound to the hybrid Chernoff bound [13] is $r = \frac{\frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{1-\delta\beta}{2})}{-2\epsilon^2}}{\frac{3\tau}{\epsilon^2}\ln(\frac{2}{1-\delta\beta})} = \frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau}$. The regime classification boundaries $\hat{\sigma}_1$ and $\hat{\sigma}_2$ are found by solving $\frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau} = 1$, yielding $\hat{\sigma}_1 = \frac{1-\sqrt{1-24\tau}}{2}$ and $\hat{\sigma}_2 = \frac{1+\sqrt{1-24\tau}}{2}$ for $\tau \leq \frac{1}{24}$. The condition $\frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau} < 1$ ensures that the variance-augmented Hoeffding-Bernstein inequality provides a tighter lower bound. Thus, the bound $N = \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2}$ holds for the intervals $0 \leq \hat{\sigma} \leq \hat{\sigma}_1$ and $\hat{\sigma}_2 < \hat{\sigma} \leq 1$ when $\tau \leq \frac{1}{24}$, and for $0 \leq \hat{\sigma} \leq 1$ when $\tau > \frac{1}{24}$, respectively. In other cases, the hybrid Chernoff bound $N = \frac{3\tau}{\epsilon^2}\ln(\frac{2}{1-\delta\beta})$ is selected.

(2) Support threshold. The support $\hat{\sigma}'$ in sample r_s is equal to the normalized support $\hat{\sigma}$ in r for random sampling, i.e., $\hat{\sigma}' = \hat{\sigma}$.

(3) Confidence threshold. We adjust the confidence threshold in the samples to $\delta' = \frac{1-\frac{\epsilon}{4}}{1+\frac{\epsilon}{4}}\delta$. Please refer to [1] for details.

Example 4: *Consider sampling with discovery parameters $\hat{\sigma} = 0.01, \epsilon = 0.001, \delta = 0.999, \tau = 0.02, \beta = 1$. First, we compute the regime boundary: $\hat{\sigma}_1 = \frac{1-\sqrt{1-24\times0.02}}{2} \approx 0.14$.As $\hat{\sigma} = 0.01 < \hat{\sigma}_1$, the sampling size is determined by the variance-augmented bound: $N = \frac{\hat{\sigma}(1-\hat{\sigma})\ln\left(\frac{2}{1-\delta\beta}\right)}{2\epsilon^2} = \frac{0.01\times0.99\times\ln(2000)}{2\times0.001^2} \approx 37,624$. This is tighter than the hybrid Chernoff bound: $N = \frac{3\tau}{\epsilon^2}\ln\left(\frac{2}{1-\delta\beta}\right) = \frac{3\times0.02}{0.001^2}\ln(2000) \approx 456,055$. This comparison demonstrates that by incorporating the binomial variance $Var(X)$ for* CFDs*, our approach reduces the required sample size by over 90% compared to the Chernoff-based baseline.*

Algorithm 2 RepSampler (Similarity-based Stratified Sampling)

Input: Relation r, thresholds σ, δ, ϵ, Chernoff coefficient τ, LSH parameters L, zero-one encoding matrix M
Output: Representative sample set S
$N \leftarrow$ calculate sampling lower bound via Theorem 2
$\mathcal{B} = \{B_1, \ldots, B_k\} \leftarrow$ Partition M via LSH $h(\cdot)$ using L
$S \leftarrow \emptyset, \mathcal{B} \leftarrow$ sort buckets in $\mathcal{B}$ in ascending order of size $|B_i|$
while $|S| < N$ **and** $\mathcal{B}$ is nonempty **do**
  for each bucket $B_i \in \mathcal{B}$ **do**
    if B_i is nonempty **then**
      $s \leftarrow$ randomly sample a tuple from B_i
      $S \leftarrow S \cup \{s\}, \quad B_i \leftarrow B_i \setminus \{s\}$
      if $|S| = N$ **then**
        break
    else
      $\mathcal{B} \leftarrow \mathcal{B} \setminus \{B_i\}$
return S

Remark. Compared to previous sampling methods, our sampling strategy exhibits the following characteristics. (1) Optimized bound tightness. By incorporating the binomial variance $Var(X) = N\hat{\sigma}(1-\hat{\sigma})$ into the concentration inequality, our variance-augmented lower bound N is tighter than the non-parametric Hoeffding and hybrid Chernoff bounds, particularly for low-support CFDs (i.e., small $\hat{\sigma}$) commonly found in large-scale relations. (2) Dimensional scalability. The derived lower bound N is independent of the number of tuples n. This ensures that the computational overhead of CFD discovery remains limited even as the dataset scales to millions of rows.

7.2 Representative Tuple Sampling

Existing sampling methods often overlook low-support CFDs. To address this, we perform stratified sampling on the zero-one matrix M generated in Section 5.1, so that CFDs with low support are represented within smaller buckets. We then sample tuples from each bucket according to the sampling bound N derived in Section 7.1 to construct the set S. As a result, S satisfies the sampling guarantees while preserving the diversity of CFDs.

Similarity-based Representative Sampling. Since each row $m_i \in M$ is a concatenation of zero-one vectors, the inner product $m_i \cdot m_j$ counts the number of identical attribute values between tuples t_i and t_j. To ensure that both high- and low-support CFDs can be discovered, we propose similarity-aware stratified sampling. We employ locality sensitive hashing (LSH) [41] to group the rows of M into collision buckets based on the number of identical attribute values, facilitating representative sampling over the original relation r.

As shown in Algorithm 2, RepSampler proceeds as follows to ensure the discovery of diverse CFDs. For the zero-one matrix M, RepSampler employs locality sensitive hashing (LSH) [41] based

Algorithm 3 PCFDFinder (Data-parallel CFD mining)

1: **Input:** Correlated attribute sets $Corr = \{Corr_1, Corr_2, \ldots, Corr_l\}$, the representative tuple set S, a sequential CFD mining algorithm Λ, support σ, confidence δ, and the relation r
2: **Output:** A minimal CFD cover Σ_s satisfying thresholds
3: $R_{sub} = \{r_1, r_2, \ldots, r_l\} \leftarrow$ Generate from $Corr$ and S
4: Create l threads across multiple processors
5: **for** each thread t_i for $i \in [1, l]$ executing in parallel **do**
6: $\quad \Sigma_{si} \leftarrow$ Mine the cover of *minimal* CFDs from r_i using Λ
7: $\quad$ **for** each CFD φ in Σ_{si} **do**
8: $\quad\quad$ Verify the CFD φ on r and remove any instances of φ from Σ_{si} that violate r.
9: $\Sigma_s \leftarrow \Sigma_{s1} \cup \Sigma_{s2} \cup \ldots \cup \Sigma_{sl}$
10: **return** Σ_s

on the MinHash strategy [10] to map tuples into buckets $\mathcal{B} = \{B_1, \ldots, B_k\}$ (Line 4). Unlike density clustering [21], RepSampler converges efficiently by leveraging the MinHash strategy, which processes only the non-zero indices in each row, thereby avoiding unnecessary computation over zero entries. For each row m_i, RepSampler generates hash signatures in a single scan. The LSH mechanism ensures that tuples within the same bucket B_i share high Jaccard similarity [10] with high probability, indicating that they satisfy the same CFDs. The time complexity of this stage is $O(n)$ [41], where n is the number of tuples.

RepSampler first removes any bucket B_i with $|B_i| < \sigma$, as such buckets cannot satisfy the support threshold of a CFD. It then processes the remaining buckets in ascending order of size, selecting one tuple from each non-empty bucket in each round. During this iterative process, buckets are only discarded once all their tuples have been selected, until the total sample size $|S|$ reaches the bound N (Lines 5–15). This strategy ensures that low-support CFD patterns in small buckets are preserved in the sample set S, preventing them from being ignored by high-support CFD patterns in large buckets. The number of samples selected across all buckets is constrained by the sampling lower bound N derived in Theorem 2 (Line 3). RepSampler preserves low-support CFDs by sampling from similarity-based buckets.

Example 5: *In Table 1,* RepSampler *first partitions M into similarity-based buckets* $\mathcal{B} = \{\{t_5\}, \{t_3, t_4\}, \{t_1, t_2, t_6, t_7\}\}$ *and computes* $N = 5$. *Following the sampling strategy, buckets are sorted by size (assuming all buckets here meet* σ *and are retained). (1) In the first iteration,* t_5, t_3, t_2 *are randomly sampled from each bucket, resulting in* $S = \{t_5, t_3, t_2\}$. *(2) Since* $|S| = 3 < N$, *the second iteration continues with the remaining non-empty buckets.* t_4 *and* t_7 *are randomly selected, bringing the total count to* $|S| = 5$. *The process terminates and returns* $S = \{t_5, t_3, t_2, t_4, t_7\}$, *successfully preserving the rare bucket* $\{t_5\}$.

Complexity Analysis. The total time complexity of RepSampler is linear in the number of tuples, i.e., $O(n)$, with the number of attributes and hash functions treated as constants. Specifically, RepSampler scales linearly by integrating three stages: (1) hash-based partitioning via LSH, (2) bucket management, and (3) stratified sampling to extract N samples (see [1] for more details).

8 PARALLEL RULE DISCOVERY

We propose PCFDFinder, a CFD discovery framework that leverages correlated attribute sets (Section 6) and representative tuples (Section 7) to enable the parallel execution of existing CFD discovery algorithms via data parallelism. First, we introduce PCFDFinder, which uses sub-tables as parallel inputs for advanced CFD mining algorithms (Section 8.1). Then, we analyze how PCFDFinder reduces computational overhead (Section 8.2).

8.1 Parallel Discovery Framework

To facilitate parallel discovery, we decompose the representative tuple set S (obtained in Section 7.2) into k sub-tables $\{r_{s1}, \ldots, r_{sk}\}$. This decomposition is guided by the correlated attribute sets $Corr = \{corr_1, \ldots, corr_k\}$ derived in Section 6.3, where each sub-table r_{si} is generated by projecting S onto the corresponding attribute set $corr_i$ (i.e., $r_{si} = \pi_{corr_i}(S)$). We then define cover $\Sigma_s = \bigcup \Sigma_{si}$, where each cover Σ_{si} is the set of minimal CFDs discovered from the corresponding sub-table r_{si}.

Next, we introduce PCFDFinder, a parallel framework for mining CFDs. As outlined in Algorithm 3, PCFDFinder aims to discover the minimal cover Σ_s of CFDs from r in parallel (line 2). PCFDFinder projects the representative tuple set S onto correlated attribute sets $Corr = \{corr_1, corr_2, \ldots, corr_l\}$, and obtains l input sub-tables $R_{sub} = \{r_1, r_2, \ldots, r_l\}$ (Line 3). Then, PCFDFinder spawns l threads to concurrently run a sequential CFD discovery algorithm Λ on each sub-table r_i (Lines 4-6). Within each thread, every discovered candidate CFD φ is immediately validated against the original instance r with a complexity of $O(n)$, where n is the number of tuples in r, and CFDs violating r are discarded from Σ_{si} (Lines 7-8). Once all threads complete, PCFDFinder computes the union of the validated local covers: $\Sigma_s = \bigcup_{i=1}^{l} \Sigma_{si}$ (Line 9). The final cover Σ_s is returned as the cover of CFDs on relation r (Line 10).

8.2 The reduction of computation

The time complexity of the sequential CFD mining algorithm is $O(n \times m \times d^m)$ [62], where n, m and d represent the number of tuples, attributes, and the average domain size of attributes, respectively. With data parallelism in PCFDFinder, this time complexity is reduced to $O(n_i \times m_i \times {d_i}^{m_i})$, where n_i, m_i, and d_i are the number of tuples, attributes, and domain size in the largest sub-table. As a result, CFD discovery efficiency is improved by a factor of $\eta = O(\frac{n \times m \times d^m}{n_i \times m_i \times d_i^{m_i}}) \geq O(\frac{d^{(1-\kappa_2)\times m}}{\kappa_1 \times \kappa_2})$, where $d_i \leq d$, and κ_1, κ_2 denote the ratios of tuple counts and attribute counts in the largest sub-table relative to the original relation, respectively. In parallel execution, PCFDFinder delivers linear speedup via the tuple scaling factor κ_1^{-1} and exponential efficiency gains via the attribute reduction factor κ_2^{-1} across decomposed sub-tables.

9 EXPERIMENTAL STUDY

In this section, we experimentally evaluated (1) the efficiency of SCFDM; (2) the scalability of SCFDM; (3) the effectiveness (discovery accuracy) of SCFDM; (4) sensitivity to CFD core parameters; (5) the ablation study of SCFDM; and (6) a case study of CFDs on real-world and synthetic datasets.

9.1 Experimental Setting

Datasets. As shown in Table 2, we utilized five real-world datasets (1-5) sourced from UCI [3] and Kaggle [60], and two synthetic

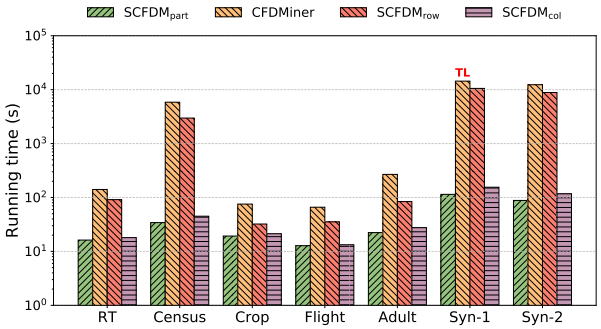
Table 2: Summary of Datasets and Algorithm Outputs

Dataset	Source	#Tuples	#Attributes	#CFDs	#Sub-tables
RT	UCI	123,117	85	50,498	178
Census	UCI	199,523	45	84,309	236
Crop	Kaggle	325,835	175	78,232	412
Flight	Kaggle	1,000,000	25	6,224	104
Adult	UCI	47,621	14	435	52
Syn-1	bnlearn	20,000	48	2,263	214
Syn-2	bnlearn	20,000	37	1,755	152

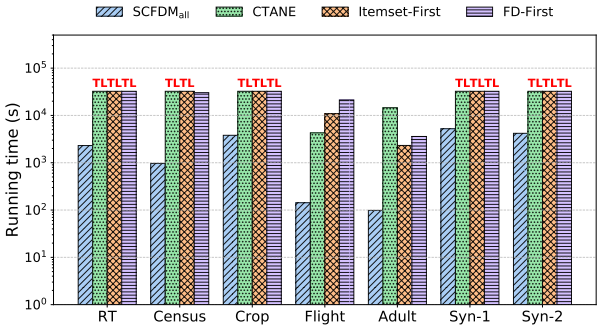
datasets (6-7) generated by Bayesiannetwork [64]: (1) RT-IOT (RT), a dataset integrating IoT devices and sophisticated network attack methodologies. (2) Census-Income (Census), a dataset containing weighted census data extracted from the 1994 and 1995 current population surveys. (3) Crop, a dataset combining optical and PolSAR remote sensing images for field classification. (4) Flight, a dataset providing data on the on-time performance of domestic flights operated by major U.S. airlines. (5) Adult, a dataset predicting whether income exceeds $50K/year based on census data. (6) and (7) are two Bayesian-based synthetic datasets: Synthetic-1 (Syn-1) and Synthetic-2 (Syn-2), each created using methods outlined in [75]. The "#Sub-tables" column in Table 2 reports the number of partitioned sub-tables generated by SCFDM for each dataset.

Algorithms. We implemented eight CFD discovery algorithms in C++. (1) CFDMiner [26]: the state-of-the-art algorithm for mining constant CFDs. (2) CTANE [26]: an advanced level-wise CFD mining algorithm that discovers both constant and variable CFDs. (3) Itemset-First [62]: an advanced hybrid algorithm for mining both constant and variable CFDs, which first mines itemsets and then discovers FDs corresponding to these itemsets using the depth-first search. (4) FD-First [62]: an advanced hybrid algorithm for mining both constant and variable CFDs, which first discovers FDs and then mines itemsets using the FD-first strategy. (5) $\text{SCFDM}_{\text{part}}$: the proposed algorithm for discovering only constant CFDs, which incorporates AttrFinder for correlated attribute set extraction, RepSampler for sampling, and PCFDFinder (utilizing CFDMiner [26] as the sequential algorithm Λ) for data-parallel CFD discovery. (6) $\text{SCFDM}_{\text{all}}$: the proposed algorithm designed for mining both constant and variable CFDs, where Itemset-First algorithm [62] serves as the sequential algorithm Λ of PCFDFinder. (7) $\text{SCFDM}_{\text{row}}$ and (8) $\text{SCFDM}_{\text{col}}$: two variants of $\text{SCFDM}_{\text{part}}$, where the first omits AttrFinder and the second omits RepSampler. The CFDMiner [26] algorithm is employed as the base sequential discoverer Λ within $\text{SCFDM}_{\text{row}}$ and $\text{SCFDM}_{\text{col}}$.

Ground truth of CFDs. For real-world datasets, we employ a dual-mining strategy within Metanome [55] to ensure comprehensive coverage. Specifically, the level-wise algorithm CTANE [26] is used for exhaustive CFD discovery, while CFDMiner [26] is applied to supplement the discovery of constant CFDs. For synthetic datasets, the ground truth Σ is inherently defined by the Bayesian generative process. We obtain the target CFD constraints as structural priors during the data synthesis phase, ensuring that the generated instances strictly adhere to the predefined dependency logic. Following the automated phase, a manual expert review is conducted to filter out redundant rules or CFDs that deviate from the formal



(a) Running time comparison of $\text{SCFDM}_{\text{part}}$ and baselines



(b) Running time comparison of $\text{SCFDM}_{\text{all}}$ and baselines

Figure 3: Efficiency evaluation of SCFDM and baselines

definitions. This rigorous process yields a high-fidelity "gold standard" Σ for each dataset. The total number of ground truth CFDs for each dataset is recorded in the "#CFDs" column of Table 2.

Accuracy metrics. Let r be the original relation and r_s be the set of sampled sub-tables partitioned from r. Let Σ and Σ_s denote the ground-truth CFDs discovered from the original relation r and the parallel discovery results obtained from r_s, respectively. We adopt the following metrics to evaluate the effectiveness of SCFDM: (1) Precision (P) measures the proportion of discovered CFDs in Σ_s that are also valid and minimal on the original relation r: $P(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma_s|}$; (2) Recall ($R$) measures the proportion of the total ground-truth CFDs from the original relation r that are successfully recovered from the sampled sub-tables r_s: $R(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma|}$; (3) F1-score is the harmonic mean of P and R, providing a balanced evaluation of the discovery performance: $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$.

Default settings. We conducted experiments on a cluster with four nodes, each equipped with two 28-core 2.70GHz CPUs. Each experiment was performed 3 times, and the average result is reported. Unless stated otherwise, we set the following parameters: normalized support threshold $\hat{\sigma} = 0.2$, confidence threshold $\delta = 0.95$, support error bound $\epsilon = 0.005$, Chernoff ratio $\tau = 0.05$, recall $\beta = 0.9$. The system uses 8 CPUs with 120 cores, 224GB of memory, and a 4-hour time limit for constant CFD discovery (extended to 9 hours for discovering both constant and variable CFDs).

9.2 Experimental Results

Exp-1: Efficiency of SCFDM. We evaluate the efficiency of SCFDM by comparing it with state-of-the-art baselines across all seven datasets. Figure 3 illustrates the execution time, where "TL"

Table 3: Parallel performance of SCFDM with varying $|q|$

Algorithm	Dataset	Execution Time (s)			
		$\|q\| = 28$	$\|q\| = 56$	$\|q\| = 84$	$\|q\| = 112$
$SCFDM_{part}$	RT	40.320	25.257	19.574	16.233
$SCFDM_{all}$	Census	2020.923	1425.339	1112.194	968.722
$SCFDM_{all}$	Adult	241.963	145.907	113.445	98.254
$SCFDM_{part}$	Syn-1	220.117	175.322	131.020	114.437

denotes that a baseline exceeded the designated time limit (14,400s for Figure 3(a) and 32,400s for Figure 3(b)).

Figure 3(a) compares $SCFDM_{part}$, $SCFDM_{row}$, and $SCFDM_{col}$ against CFDMiner. $SCFDM_{part}$ consistently exhibits the highest efficiency, achieving an average speed-up of 107.78× across all datasets. Notably, on the Syn-1 dataset, while CFDMiner failed to complete within the time limit, $SCFDM_{part}$ finished in only 114.44s, representing a performance gain of at least 125.83×. The most significant improvement was observed on Census, where $SCFDM_{part}$ outperformed CFDMiner by 171.14×.

Figure 3(b) evaluates $SCFDM_{all}$ against CTANE, Itemset-First, and FD-First. $SCFDM_{all}$ maintains a substantial lead, especially on complex datasets. On average, $SCFDM_{all}$ is at least 9.94× faster than the best-performing baseline for each dataset. Note that for datasets where baselines exceeded the time limit, we conservatively recorded the time as 32,400s (TL); the actual execution times are likely significantly higher. Specifically, it achieved a maximum speed-up of 148.10× on the Adult dataset compared to CTANE. On RT, Census, and Crop, where all three baselines triggered the time limit, $SCFDM_{all}$ successfully completed the task, demonstrating its superior scalability in handling massive search spaces.

Analysis. The efficiency of SCFDM stems from two primary factors. First, by partitioning the original relation r into sub-tables for parallel discovery of CFDs, the algorithm effectively prunes the search space and reduces the computational overhead of each individual task. Second, the integration of AttrFinder and RepSampler allows SCFDM to quickly extract correlated vector sets and representative tuples, bypassing the exhaustive attribute-pair checks that bottlenecked the baselines. In contrast, baseline algorithms (e.g., CTANE and FD-First) struggled with exponential search space explosions because they lacked an early-stage pruning mechanism for irrelevant attributes in relation r.

Exp-2: Scalability of SCFDM. We evaluated the scalability of SCFDM by varying (1) the tuple sampling ratio $|r|$, (2) the number of attributes $|R|$, (3) the normalized support threshold $|\hat{\sigma}|$, (4) the confidence threshold $|\delta|$, (5) the maximum CFD size $|k|$, which limits the max number of predicates in a CFD, and (6) the number of CPU cores $|q|$ across multiple nodes. Figures 4(a), 4(d), 4(g) and 4(j) compare $SCFDM_{part}$, $SCFDM_{row}$, and $SCFDM_{col}$ with CFDMiner, which is faster than other baselines by an order of magnitude since it only mines constant CFDs, while other baselines mine both constant and variable CFDs [26]. For this reason, in Figures 4(b), 4(c), 4(e), 4(f), 4(h), 4(i), 4(k) and 4(l), we excluded CFDMiner from comparisons between $SCFDM_{all}$, CTANE, Itemset-First, and FD-First.

Varying $|r|$. As shown in Figures 4(a), 4(b) and 4(c), the tuple sampling ratio $|r|$ was tuned from 0.6 to 1.0 on Crop and from 0.1 to 0.5 on Flight and RT. As $|r|$ increased, the running times of $SCFDM_{part}$, $SCFDM_{all}$, $SCFDM_{row}$, $SCFDM_{col}$, and all four baselines also increased. On average, $SCFDM_{part}$ achieved a 3.47-fold efficiency gain on Crop. Meanwhile, $SCFDM_{all}$ exhibited even more substantial improvements, outperforming the baselines by 41.95 times and 23.24 times on Flight and RT, respectively.

Table 4: Effectiveness of CFD Discovery

Dataset	Metric	CFDMiner	CTANE	Itemset-First	FD-First	SCFDM
RT	Precision	0.985	0.990	0.978	0.979	0.986
	Recall	0.075	0.323	0.526	0.447	0.960
	F_1	0.139	0.487	0.684	0.614	**0.973**
Census	Precision	0.982	0.970	0.966	0.966	0.987
	Recall	0.062	0.610	0.828	1.000	0.983
	F_1	0.117	0.749	0.892	0.983	**0.985**
Crop	Precision	0.970	0.963	0.950	0.956	0.971
	Recall	0.084	0.210	0.455	0.432	0.933
	F_1	0.155	0.345	0.615	0.595	**0.952**
Flight	Precision	0.999	0.996	0.995	0.995	0.997
	Recall	0.140	0.998	0.999	0.999	0.998
	F_1	0.246	0.997	0.997	0.997	**0.997**
Adult	Precision	1.000	1.000	1.000	1.000	1.000
	Recall	0.338	1.000	1.000	1.000	1.000
	F_1	0.505	1.000	1.000	1.000	**1.000**
Syn-1	Precision	0.975	0.980	0.964	0.966	0.979
	Recall	0.024	0.116	0.301	0.345	0.953
	F_1	0.047	0.207	0.459	0.508	**0.966**
Syn-2	Precision	0.985	0.979	0.974	0.974	0.980
	Recall	0.079	0.466	0.550	0.557	0.966
	F_1	0.146	0.631	0.703	0.709	**0.973**

Results obtained before reaching the time (9 hours)/memory (224GB) limit.

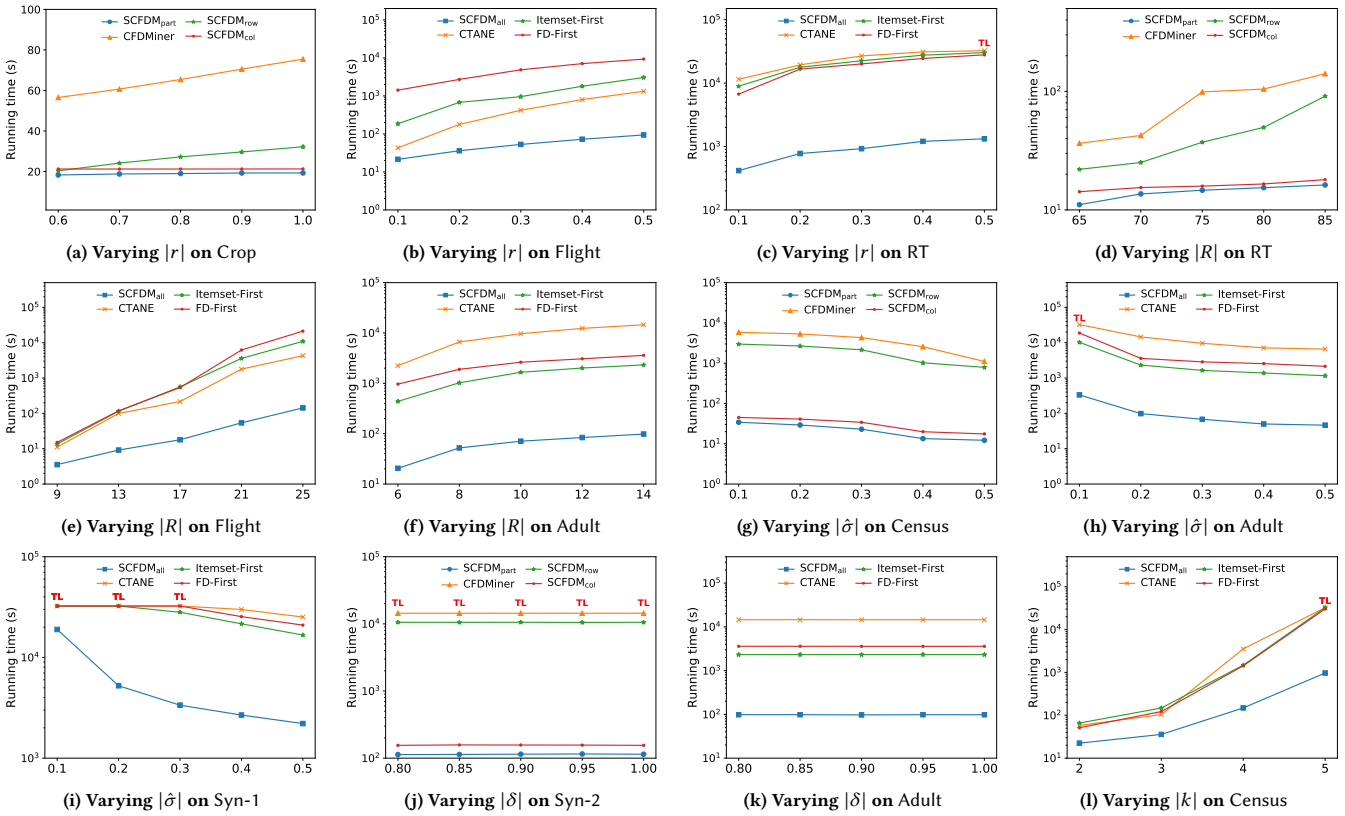
Varying $|R|$. Figures 4(d)–(f) show the impact of varying the number of attributes $|R|$ across three datasets: RT (65–85), Flight (9–25), and Adult (6–14). As $|R|$ increased, the runtimes of all baselines escalated significantly. In contrast, our methods ($SCFDM_{part}$, $SCFDM_{row}$, $SCFDM_{col}$, and $SCFDM_{all}$) exhibited much more resilient and gradual growth. Notably, the SCFDM series achieved substantial efficiency gains, outperforming baselines by an average of 5.96× on RT, 72.92× on Flight, and 66.57× on Adult. Figure 4(d) highlights that $SCFDM_{col}$ outperforms $SCFDM_{row}$ in speedup.

Varying $|\hat{\sigma}|$. Figures 4(g)–(i) illustrate the impact of varying the normalized support threshold $|\hat{\sigma}|$ (as defined in Section 7.1) from 0.1 to 0.5 across Census, Adult and Syn-1. As $|\hat{\sigma}|$ increased, all baselines saw a significant decrease in execution time due to more candidate pruning. Nevertheless, $SCFDM_{part}$ and $SCFDM_{all}$ consistently outperformed the baselines, delivering average speedups of 171.13×, 65.47×, and 4.39× on Census, Adult and Syn-1, respectively.

Varying $|\delta|$. As shown in Figures 4(j) and 4(k), we found that the execution times of SCFDM, CTANE, Itemset-First, and FD-First showed little change when varying the confidence threshold $|\delta|$ from 0.80 to 1.00. This is because $|\delta|$ has a limited effect on the search space of CFDs. Specifically, SCFDM achieved an average speedup of 126.05× on Syn-2 and 69.49× on Adult, respectively.

Varying $|k|$. We varied the maximum CFD size $|k|$ from 2 to 5 on the Census dataset. Figure 4(l) shows that as $|k|$ increased, the running times of $SCFDM_{all}$, CTANE, Itemset-First, and FD-First grew exponentially, with $SCFDM_{all}$ growing more slowly. On average, $SCFDM_{all}$ was 28.96-fold more efficient.

Varying $|q|$. We increased the number of CPU cores $|q|$ from 28 to 112 to assess the runtime reduction of SCFDM on the RT, Census, Adult, and Syn-1 datasets. Table 3 shows that SCFDM scales efficiently with more CPU cores. As $|q|$ increased, the runtime of



(a) Varying $|r|$ on Crop (b) Varying $|r|$ on Flight (c) Varying $|r|$ on RT (d) Varying $|R|$ on RT

(e) Varying $|R|$ on Flight (f) Varying $|R|$ on Adult (g) Varying $|\hat{\sigma}|$ on Census (h) Varying $|\hat{\sigma}|$ on Adult

(i) Varying $|\hat{\sigma}|$ on Syn-1 (j) Varying $|\delta|$ on Syn-2 (k) Varying $|\delta|$ on Adult (l) Varying $|k|$ on Census

Figure 4: Scalability evaluation of SCFDM

SCFDM decreased from 630.83s to 299.41s on average. Specifically, $\text{SCFDM}_{\text{part}}$ was 1.99-fold faster on the RT and Syn-1 datasets, while $\text{SCFDM}_{\text{all}}$ was 2.12-fold faster on Census and Adult on average.

Analysis. The experimental results show that SCFDM handles large datasets with hundreds of thousands of tuples and nearly a hundred attributes effectively. (1) Its execution time varies most with $|R|$ and $|k|$, due to their exponential effect on the search space of sub-tables. (2) SCFDM scales well with parameters $|r|$ and $|\hat{\sigma}|$ by sampling a fixed number of representative tuples and efficiently discovering CFDs on sub-tables. (3) The parameter $|\delta|$ has little impact on the search space. (4) Additionally, increasing the number of CPU cores $|q|$ speeds up SCFDM by distributing the workload across CPU cores with low communication overhead.

Exp-3: Effectiveness of SCFDM**.** We evaluated SCFDM's effectiveness on all seven datasets. As shown in Table 4, SCFDM consistently achieves the highest F_1-score across all datasets. While all methods generally maintain high precision, their recall performance reveals significant differences. Specifically, CFDMiner exhibits the lowest recall (e.g., only 0.024 on Syn-1); although it operates with relatively high efficiency, it is inherently limited to discovering constant CFDs, thus missing a vast number of CFDs containing wildcards "_". For other baselines such as CTANE and FD-First, the low recall is primarily due to their high computational complexity. These methods frequently trigger the time (9 hours) or memory (224GB) limits on complex datasets, allowing them to output only a small fraction of the total results. In contrast, SCFDM strikes a superior balance by efficiently handling wildcards "_" while maintaining scalability. Notably, on the RT and Syn-1 datasets, SCFDM dramatically improves the F_1-score to 0.973 and 0.966, respectively, significantly outperforming all existing baselines.

Analysis. The high F_1-scores across all seven datasets highlight SCFDM's effectiveness in mining CFDs. This success stems from two main advantages: First, unlike CFDMiner, which is restricted to constant CFDs, SCFDM effectively captures complex dependencies with PCFDFinder, ensuring high precision and recall. Second, while other baselines (e.g., CTANE, FD-First) struggle with state-space explosion and premature termination on large-scale or high-dimensional data, AttrFinder and RepSampler of SCFDM bypass these computational bottlenecks, consistently identifying a near-complete set of CFDs within the resource limits.

Exp-4: Sensitivity to CFD **Core Parameters.** We evaluated the impact of core CFD parameters (support thresholds $|\hat{\sigma}|$) on the RT and Syn-1 datasets based on the F_1-score metric. As shown in Table 5, SCFDM demonstrates exceptional robustness to variations in support thresholds. Specifically, when the threshold is small (e.g., $|\hat{\sigma}| = 0.10$), the F_1-scores of all baselines drop sharply, with some falling below 0.11 (e.g., 0.054 for CFDMiner and 0.102 for CTANE on RT). This is because lower support thresholds expand the search space, causing baselines to terminate prematurely and miss a large number of CFDs. In contrast, SCFDM maintains a consistently high F_1-score (above 0.92) even at the most stringent settings. As $|\hat{\sigma}|$ increases to 0.50, the performance gap narrows as the search

Table 5: Impact of CFD parameters (support thresholds $|\hat{\sigma}|$) on F_1-score across different algorithms

Dataset	Algorithm	Metric	0.10	0.20	0.30	0.40	0.50
RT	CFDMiner	F_1	0.054	0.139	0.170	0.162	0.160
	CTANE		0.102	0.487	0.722	0.954	0.980
	Itemset-First		0.198	0.684	0.804	0.959	0.979
	FD-First		0.220	0.614	0.794	0.959	0.979
	SCFDM		**0.924**	**0.973**	**0.970**	**0.974**	**0.980**
Syn-1	CFDMiner	F_1	0.042	0.047	0.054	0.102	0.097
	CTANE		0.097	0.207	0.455	0.707	0.970
	Itemset-First		0.215	0.459	0.662	0.774	0.966
	FD-First		0.177	0.508	0.725	0.781	0.969
	SCFDM		**0.949**	**0.966**	**0.969**	**0.969**	**0.970**

Results obtained before reaching the time (9 hours)/memory (224GB) limit.

space becomes more manageable for baselines; however, SCFDM still yields the best or competitive results across all cases. These results underscore that SCFDM is significantly more reliable than existing methods when the support threshold is low or unknown, ensuring high discovery accuracy regardless of parameter settings.

Exp-5: Ablation Study of SCFDM. In our ablation study, we investigated the effectiveness of the core components AttrFinder, RepSampler, and PCFDFinder of SCFDM on Crop. In Table 6, all three components are indispensable for the model's performance, as removing any single module causes the runtime to exceed the 9-hour limit and leads to a significant degradation in discovery quality. Specifically, the absence of AttrFinder causes the F_1-score to plummet from 95.2% to 67.9% with a recall of only 52.2%, proving its necessity in narrowing the attribute search space. Similarly, without RepSampler, the model fails to complete within the time limit and suffers an F_1 drop to 88.3%, highlighting the importance of representative tuple sampling in accelerating the discovery process. Notably, PCFDFinder proves to be the most critical component; its removal results in the most severe performance loss, with recall and F_1-score dropping to 26.6% and 41.8%, respectively. These results underscore that the synergy of these three components is what enables SCFDM to maintain high effectiveness while remaining computationally efficient under practical time constraints.

Exp-6: Case study on data repair. We conducted a comprehensive case study on the noisy Adult dataset to evaluate SCFDM's practical effectiveness on data quality. For example, in the "Relationship" attribute, some tuples have the value "Husband" but are incorrectly labeled as "Female" in the "Gender" attribute. Additionally, some cells contain missing values marked as "?". SCFDM identified 435 CFDs that satisfy the support threshold of 200 and the confidence threshold of 0.95, including complex rules like φ_5 : ([Education, Occupation] $\rightarrow$ Workclass, (some-college, _ | _)) and φ_6 : ([Gender, Hours, Occupation] $\rightarrow$ Marital, (male, 50, sales | married-civ-spouse)). φ_5 suggests that individuals with a "some-college" education may face workclass limitations depending on their occupation. For example, certain workclasses (e.g., federal government jobs) require a higher education level than "some-college". We can use φ_5 to correct tuples where the "Workclass" is inconsistent with the educational background. φ_6 suggests that older men around 50, who work in sales and log over 50 hours per week, are likely to be married and supporting a family. We use rule φ_6 to correct tuples where male individuals in high-intensity sales jobs are not listed as married. In contrast, exact CFD miners failed to discover φ_5 and φ_6 because they are strictly violated by noisy tuples

Table 6: Ablation study of key SCFDM components on Crop

Model Configuration	Prec. (%)	Rec. (%)	F1 (%)	Runtime (s)
SCFDM (Full Model)	97.1	93.3	95.2	3804.1
Ablations				
w/o AttrFinder	97.2	52.2	67.9	over 9 hours
w/o RepSampler	97.1	80.9	88.3	over 9 hours
w/o PCFDFinder	97.1	26.6	41.8	over 9 hours

Results obtained before reaching the time (9 hours)/memory (224GB) limit.

(e.g., "Husband" labeled as "Female" or missing values "?"). Furthermore, while SCFDM successfully extracts complex dependencies like φ_5 and φ_6, baselines such as FD-First fail to discover these patterns within a reasonable time limit (9 hours). This is mainly due to the exponential search space of CFDs, which creates a major bottleneck for traditional discovery algorithms and hinders the identification of deep-seated rules in large, noisy real-world data.

Summary. Through extensive experiments, we observe the following: (1) Superior Efficiency. SCFDM significantly outperforms state-of-the-art baselines, achieving an average speedup of over 100×. Notably, it remains efficient on complex datasets where baselines frequently trigger time limits, with a peak performance gain of 171.14× on Census. (2) Robust Scalability. SCFDM demonstrates resilient scalability across all core parameters. Unlike baselines that suffer from exponential runtime growth as $|R|$ or $|k|$ increase, SCFDM maintains a gradual and manageable growth curve. (3) High Discovery Quality. SCFDM ensures the effectiveness of CFDs, consistently achieving the highest F_1-scores (averaging above 0.96) and striking a superior balance between precision and recall where others fail. (4) Component Vitality. The ablation study confirms that AttrFinder, RepSampler, and PCFDFinder are indispensable; their synergy prevents state-space explosion and allows SCFDM to handle wildcards and large search spaces that are prohibitive for baselines. (5) Efficient Parallelism. SCFDM scales effectively with computational resources, achieving approximately a 2-fold speedup when the number of CPU cores $|q|$ increases from 28 to 112. (6) Practical Utility. The case study highlights that SCFDM discovers robust, actionable knowledge capable of repairing real-world noisy data, whereas traditional baselines fail to identify rules like φ_5 and φ_6 within a practical time limit due to exponential search space.

10 CONCLUSION

We introduce SCFDM, a novel data-parallel framework designed for the efficient discovery of CFDs on large-scale datasets. SCFDM achieves significant performance gains through several key innovations: (1) Probabilistic Representation. We represent CFDs as conditional probabilities and embed raw relations in a two-stage Transformer pipeline, enabling SCFDM to capture complex attribute correlations. (2) Attention-Driven Extraction. AttrFinder extracts correlated attribute sets from global attention, which are then used to divide the original large relation into manageable sub-tables. (3) Diversity-Aware Sampling. A sampling strategy (RepSampler) governed by a sampling lower bound to reduce input data size while ensuring discovery recall. (4) Data Parallelism. SCFDM uses correlated attribute sets and representative tuples to partition the relation into independent sub-tables for parallel CFD discovery.

Our future work will focus on extending SCFDM to support more complex rules with machine learning predicates (e.g., REEs [29]).

REFERENCES

[1] 2026. Code, datasets and full version. https://anonymous.4open.science/r/CFDs-2026-EE65/.

[2] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling relational data a survey. *The VLDB Journal* 24 (2015), 557–581.

[3] Arthur Asuncion, David Newman, et al. 2007. UCI machine learning repository.

[4] George Bennett. 1962. Probability inequalities for the sum of independent random variables. *J. Amer. Statist. Assoc.* 57, 297 (1962), 33–45.

[5] H Russell Bernard, Amber Wutich, and Gery W Ryan. 2016. *Analyzing qualitative data: Systematic approaches*. SAGE publications.

[6] George Beskales, Ihab F Ilyas, Lukasz Golab, and Artur Galiullin. 2014. Sampling from repairs of conditional functional dependency violations. *The VLDB Journal* 23 (2014), 103–128.

[7] Tobias Bleifuß, Susanne Bülow, Johannes Frohnhofen, Julian Risch, Georg Wiese, Sebastian Kruse, Thorsten Papenbrock, and Felix Naumann. 2016. Approximate discovery of functional dependencies for large datasets. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 1803–1812.

[8] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient denial constraint discovery with hydra. *Proceedings of the VLDB Endowment* 11, 3 (2017), 311–323.

[9] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780199535255.001.0001

[10] Andrei Z Broder. 1997. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*. IEEE, 21–29.

[11] Venkatesan T Chakaravarthy, Vinayaka Pandit, and Yogish Sabharwal. 2009. Analysis of sampling techniques for association rule mining. In *Proceedings of the 12th international conference on database theory*. 276–283.

[12] Fei Chiang and Renée J Miller. 2008. Discovering data quality rules. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1166–1177.

[13] Fan Chung and Linyuan Lu. 2002. Connected components in random graphs with given expected degree sequences. *Annals of combinatorics* 6, 2 (2002), 125–145.

[14] Zhang Chunsheng and Diao Yufeng. 2015. Conditional functional dependency discovery and data repair based on decision tree. In *2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*. IEEE, 864–868.

[15] Edgar F Codd et al. 1972. *Relational completeness of data base sublanguages*. Citeseer.

[16] Gonçalo M Correia, Vlad Niculae, and André FT Martins. 2019. Adaptively sparse transformers. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2174–2184.

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[18] Jay L Devore. 2000. Probability and statistics. *Pacific Grove: Brooks/Cole* (2000).

[19] Thierno Diallo, Nöel Novelli, and Jean-Marc Petit. 2012. Discovering (frequent) constant conditional functional dependencies. *International Journal of Data Mining, Modelling and Management* 4, 3 (2012), 205–223.

[20] Yuefeng Du, Derong Shen, Tiezheng Nie, Yue Kou, and Ge Yu. 2017. Discovering context-aware conditional functional dependencies. *Frontiers of Computer Science* 11 (2017), 688–701.

[21] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.

[22] Wenfei Fan. 2008. Dependencies revisited for improving data quality. In *Proceedings of the twenty-seventh ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 159–170.

[23] Wenfei Fan. 2015. Data quality From theory to practice. *Acm Sigmod Record* 44, 3 (2015), 7–18.

[24] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *TODS* (2008).

[25] Wenfei Fan, Floris Geerts, Jianzhong Li, and Ming Xiong. 2010. Discovering conditional functional dependencies. *IEEE Transactions on Knowledge and Data Engineering* 23, 5 (2010), 683–698.

[26] Wenfei Fan, Floris Geerts, Jianzhong Li, and Ming Xiong. 2011. Discovering conditional functional dependencies. *TKDE* 23, 5 (2011), 683–698.

[27] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2022. Parallel Rule Discovery from Large Datasets by Sampling. In *Proceedings of the 2022 International Conference on Management of Data*. 384–398.

[28] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–29.

[29] Wenfei Fan, Chao Tian, Yanghao Wang, and Qiang Yin. 2021. Parallel discrepancy detection and incremental detection. *Proceedings of the VLDB Endowment* 14, 8 (2021), 1351–1364.

[30] Ronald A Fisher. 1949. The design of experiments. (1949).

[31] Hector Garcia-Molina, Jeffrey D Ullman, and Jennifer Widom. 2000. *Database system implementation*. Vol. 672. Prentice Hall Upper Saddle River.

[32] Eve Garnaud, Nicolas Hanusse, Sofian Maabout, and Noël Novelli. 2014. Parallel mining of dependencies. In *2014 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, 491–498.

[33] Bart Goethals, Wim Le Page, and Heikki Mannila. 2008. Mining association rules of simple conjunctive queries. In *Proceedings of the 2008 SIAM International Conference on Data Mining*. SIAM, 96–107.

[34] Lukasz Golab, Howard Karloff, Flip Korn, Divesh Srivastava, and Bei Yu. 2008. On generating near-optimal tableaux for conditional functional dependencies. *Proceedings of the VLDB Endowment* 1, 1 (2008), 376–390.

[35] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in neural information processing systems* 34 (2021), 18932–18943.

[36] Wassily Hoeffding. 1994. Probability inequalities for sums of bounded random variables. *The collected works of Wassily Hoeffding* (1994), 409–426.

[37] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. Tabtransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678* (2020).

[38] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. (12 2020). https://doi.org/10.48550/arXiv.2012.06678

[39] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1999. TANE: An efficient algorithm for discovering functional and approximate dependencies. *The computer journal* (1999).

[40] Saxena H Golab L Ilyas IF. 2019. Distributed implementations of dependency discovery algorithms. *Proc. VLDB Endow* 12, 11 (2019), 1624.

[41] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.

[42] Sijia Jiang, Zijing Tan, Jiawei Wang, Zhikang Wang, and Shuai Ma. 2023. Guided conditional functional dependency discovery. *Information Systems* 114 (2023), 102158.

[43] Jan Kossmann, Thorsten Papenbrock, and Felix Naumann. 2022. Data dependencies for query optimization a survey. *The VLDB Journal* 31, 1 (2022), 1–22.

[44] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *Proceedings of the VLDB Endowment* 11, 7 (2018), 759–772.

[45] Chin Yang Lee. 1961. An algorithm for path connections and its applications. *IRE transactions on electronic computers* 3 (1961), 346–365.

[46] Charles E Leiserson and Tao B Schardl. 2010. A work-efficient parallel breadth-first search algorithm (or how to cope with the nondeterminism of reducers). In *Proceedings of the twenty-second annual ACM symposium on Parallelism in algorithms and architectures*. 303–314.

[47] Jiuyong Li, Jixue Liu, Hannu Toivonen, and Jianming Yong. 2013. Effective pruning for the discovery of conditional functional dependencies. *Comput. J.* 56, 3 (2013), 378–392.

[48] Mingda Li, Hongzhi Wang, and Jianzhong Li. 2019. Mining conditional functional dependency rules on big data. *Big Data Mining and Analytics* 3, 1 (2019), 68–84.

[49] Yanrong Li and Raj P Gopalan. 2005. Stratified Sampling for Association Rules Mining. In *Artificial Intelligence Applications and Innovations: IFIP TC12 WG12. 5-Second IFIP Conference on Artificial Intelligence Applications and Innovations (AIAI2005), September 7–9, 2005, Beijing, China 2*. Springer, 79–88.

[50] Ester Livshits, Alireza Heidari, Ihab F Ilyas, and Benny Kimelfeld. 2020. Approximate Denial Constraints. *arXiv preprint arXiv:2005.08540* (2020).

[51] Ilya Loshchilov and Frank Hutter. 2017. Fixing Weight Decay Regularization in Adam. *CoRR* abs/1711.05101 (2017). arXiv:1711.05101 http://arxiv.org/abs/1711.05101

[52] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.

[53] Frank Olken. 1993. *Random sampling from databases*. Ph.D. Dissertation. Citeseer.

[54] Kamlesh Kumar Pandey and Diwakar Shukla. 2020. Stratified sampling-based data reduction and categorization model for big data mining. In *Communication and Intelligent Systems: Proceedings of ICCIS 2019*. Springer, 107–122.

[55] Thorsten Papenbrock, Tanja Bergmann, Moritz Finke, Jakob Zwiener, and Felix Naumann. 2015. Data profiling with metanome. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1860–1863.

[56] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional dependency discovery: An experimental evaluation of seven algorithms. *PVLDB* 8, 10 (2015), 1082–1093.

[57] Thorsten Papenbrock and Felix Naumann. 2016. A hybrid approach to functional dependency discovery. In *Proceedings of the 2016 International Conference on Management of Data*. 821–833.

[58] Thorsten Papenbrock and Felix Naumann. 2017. Data-driven Schema Normalization.. In *EDBT*, Vol. 17. 342–353.

[59] Eduardo HM Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *Proceedings of the VLDB Endowment* 16, 4 (2022), 684–696.

[60] Luigi Quaranta, Fabio Calefato, and Filippo Lanubile. 2021. Kgtorrent: A dataset of python jupyter notebooks from kaggle. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 550–554.

[61] Joeri Rammelaere and Floris Geerts. 2018. Explaining repaired data with CFDs. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1387–1399.

[62] Joeri Rammelaere and Floris Geerts. 2019. Revisiting conditional functional dependency discovery: Splitting the "C" from the "FD". In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2018, Dublin, Ireland, September 10–14, 2018, Proceedings, Part II 18*. Springer, 552–568.

[63] Hemant Saxena, Lukasz Golab, and Ihab F Ilyas. 2019. Distributed discovery of functional dependencies. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1590–1593.

[64] Marco Scutari. 2010. Learning Bayesian networks with the bnlearn R package. *Journal of statistical software* 35 (2010), 1–22.

[65] BS Sharmila and Rohini Nagapadma. 2023. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity* 6, 1 (2023), 41.

[66] Shaoxu Song and Lei Chen. 2009. Discovering matching dependencies. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management, CIKM 2009, Hong Kong, China, November 2-6, 2009*. ACM, 1421–1424.

[67] Shaoxu Song and Lei Chen. 2011. Differential dependencies: Reasoning and discovery. *ACM Transactions on Database Systems (TODS)* 36, 3 (2011), 1–41.

[68] Shaoxu Song, Lei Chen, and Hong Cheng. 2013. Efficient determination of distance thresholds for differential dependencies. *IEEE Transactions on knowledge and data engineering* 26, 9 (2013), 2179–2192.

[69] Robert J Tibshirani and Bradley Efron. 1993. An introduction to the bootstrap. *Monographs on statistics and applied probability* 57, 1 (1993), 1–436.

[70] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[71] Jesse Vig. 2019. A multiscale visualization of attention in the transformer model. In *Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations*. 37–42.

[72] Jeffrey S Vitter. 1985. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)* 11, 1 (1985), 37–57.

[73] Catharine Wyss, Chris Giannella, and Edward Robertson. 2001. FastFDs: A heuristic-driven, depth-first algorithm for mining functional dependencies from relation instances extended abstract. In *DaWaK*.

[74] Ying Yan, Liang Jeff Chen, and Zheng Zhang. 2014. Error-bounded sampling for analytics on big sparse data. *Proceedings of the VLDB Endowment* 7, 13 (2014), 1508–1519.

[75] Yunjia Zhang, Zhihan Guo, and Theodoros Rekatsinas. 2020. A statistical perspective on discovering functional dependencies in noisy data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 861–876.

[76] Yanchang Zhao, Chengqi Zhang, and Shichao Zhang. 2006. Efficient Frequent Itemsets Mining by Sampling. *AMT* 138 (2006), 112–7.

[77] Jinling Zhou, Xingchun Diao, Jianjun Cao, and Zhisong Pan. 2016. An optimization strategy for CFDMiner: an algorithm of discovering constant conditional functional dependencies. *IEICE TRANSACTIONS on Information and Systems* 99, 2 (2016), 537–540.

A TRAINING DETAILS IN AttrFinder

The training of AttrFinder is a self-supervised process designed to internalize structural dependencies through masked reconstruction over the tensor $\mathcal{T}$. Given a batch of B embedded tuples $\mathcal{T}_{\mathcal{B}} \in \mathbb{R}^{B\times m\times d}$, where $B \leq n$, the training procedure follows four critical phases.

Stochastic Attribute Masking. To compel the model to learn conditional dependencies $P(Y|\mathbf{X})$, we apply a masking operator $\tilde{\mathcal{T}}_{\mathcal{B}} = \text{Mask}(\mathcal{T}_{\mathcal{B}})$. For each tuple t_j in the batch, we randomly select a subset of attributes and replace their d-dimensional embeddings $\mathbf{E}_i^{(j)}$ with a learnable $[MASK]$ token. This simulates the discovery of unknown consequents at scale, forcing the model to reconstruct the masked attributes by attending to the remaining contextual attributes across diverse tuple instances.

Multi-Head Attention Interaction (Encoder). The masked batch $\tilde{\mathcal{T}}_{\mathcal{B}}$ is processed through L layers. In each layer $l \in [1, L]$, the self-attention mechanism computes interaction matrices for each tuple t_j:

$$\mathbf{A}_{l,h}^{(j)} = \text{softmax}\left(\frac{Q_{l,h}^{(j)}(K_{l,h}^{(j)})^T}{\sqrt{d_k}}\right),$$

where Q, K are linear projections of the j-th tuple's representation from the previous layer, and $\mathbf{A}_{l,h}^{(j)} \in \mathbb{R}^{m\times m}$ captures the tuple-specific correlations.

Segmented Categorical Reconstruction. The final hidden state of the L-th layer, $\mathbf{H}^{(L)} \in \mathbb{R}^{m\times d}$, is passed through a linear projection layer. For each masked attribute A_i with c_i categories, we apply a Segmented Softmax to reconstruct the original zero-one distribution:

$$\hat{y}_{i,k} = \frac{\exp(W_{out}^{(i)}\mathbf{h}_i^{(L)} + b_{out}^{(i)})_k}{\sum_{p=1}^{c_i}\exp(W_{out}^{(i)}\mathbf{h}_i^{(L)} + b_{out}^{(i)})_p}.$$

This ensures the output for each attribute segment represents a valid probability distribution, where $\hat{y}_{i,k}$ approximates the conditional probability $P(A_i = v_k \mid attr(\mathcal{R}) \setminus \{A_i\})$.

Joint Optimization and Convergence. The model parameters $\Theta = \{\Theta_{emb}, \Theta_{attn}, \Theta_{out}\}$ are optimized end-to-end. Specifically, the embedding parameters $\Theta_{emb} = \{W_{emb}^{(i)}, \mathbf{b}_{emb}^{(i)}\}_{i=1}^{m}$ are updated simultaneously with the attention and projection weights. We minimize the batch-averaged Cross-Entropy loss $\mathcal{L}$ aggregated over all masked positions across B tuples:

$$\mathcal{L} = -\frac{1}{B}\sum_{j=1}^{B}\sum_{i\in\text{Masked}_j}\sum_{k=1}^{c_i} y_{i,k}^{(j)}\log(\hat{y}_{i,k}^{(j)}),$$

where $y_{i,k}^{(j)}$ is the ground-truth label for the i-th attribute of the j-th tuple. This global objective ensures that the gradients $\nabla_\Theta\mathcal{L}$ reflect the statistical consensus of the entire relation r. During each training step, the gradients are backpropagated from the reconstruction layer through the Transformer layers to the attribute embedding layer. The update rule for the embedding parameters at iteration t using the Adam optimizer is:

$$\Theta_{emb}^{(t+1)} = \Theta_{emb}^{(t)} - \eta\cdot\text{Adam}(\nabla_{\Theta_{emb}}\mathcal{L}),$$

where η is the learning rate (e.g., $5e^{-4}$). This joint optimization ensures that the dense representations $\mathbf{E}_i$ in the latent space are specifically tuned to minimize uncertainty in attribute reconstruction. Through these iterative updates, the attention maps $\mathbf{A}$ and the embeddings Θ_{emb} co-evolve, eventually converging to a state that reflects the inherent structural and conditional functional dependencies of the relation.

B COMPLEXITY OF AttrFinder

To provide a rigorous evaluation of the system's scalability, we decompose the computational complexity of AttrFinder into three primary stages. We contrast these with traditional discovery methods to highlight the advantages of our neural approach.

Data Encoding. The initial vectorization involves mapping the discrete relation r into the tensor $\mathcal{T} \in \mathbb{R}^{n\times m\times d}$. First, scanning n tuples to identify distinct values and performing one-hot encoding takes $O(n\cdot m)$. Subsequently, projecting these sparse vectors into a d-dimensional latent space through the attribute embedding layer requires $O(n\cdot m)$, treating d as a constant. This stage is strictly linear with respect to the database size n, ensuring efficient preprocessing for large-scale relations.

Transformer-based learning. This is the most computationally intensive phase, where the model captures attribute correlations across the dataset. For each of the n tuples, the self-attention mechanism computes an $m\times m$ interaction matrix. Across all epochs and layers, the training complexity is $O(n\cdot m^2)$, treating the number of epochs, layers, and embedding dimension as constants. Although these constants affect absolute runtime, the growth remains quadratic in the number of attributes m, far more efficient than the exponential $O(2^m)$ cost of traditional search-based methods. Moreover, the operations are highly parallelizable on GPUs, enabling substantial speedups over CPU-bound sequential mining tools.

Correlated attribute sets extraction. Once the model has converged, CFD extraction becomes independent of the dataset size n. In Transformer-based learning, Global Attention Extraction computes the saliency matrix $\bar{\mathbf{A}}$ by aggregating attention maps across all n tuples and L layers, with a complexity of $O(n\cdot m^2)$ when treating L and the number of attention heads H as constants. In Step 2, Correlated Attribute Sets Identification scans $\bar{\mathbf{A}}$ row-wise to determine candidate antecedent sets $\mathbf{X}$ for all m potential consequents, requiring only $O(m^2)$ operations.

C COMPLEXITY OF RepSampler

To provide a rigorous evaluation of the sampling efficiency, we decompose the computational complexity of RepSampler into its primary operational stages. We highlight how our approach achieves linear scalability relative to the number of tuples n.

Hash-based Partitioning. This phase organizes the data into structural bucket without the overhead of quadratic comparisons. RepSampler employs an LSH framework based on the MinHash strategy to map n tuples into similarity buckets. Since the algorithm only processes the m non-zero entries (original attributes) for each of the hash functions, the complexity of this stage is $O(n)$. Unlike iterative clustering (e.g., k-means [52]) or density-based methods (e.g., DBSCAN [21]), which often scale quadratically or require multiple passes, our hashing-based approach achieves partitioning in a single linear scan, effectively bypassing the $O(n^2)$ bottleneck of pairwise distance calculations.

Bucket Management. This stage implements the logic to preserve minority patterns, which represent CFDs despite their low support. Sorting the k resulting buckets by their population size takes $O(k \log k)$ to facilitate the diversity-first selection logic. In real-world datasets, the number of buckets k is constrained by the LSH parameters and is significantly smaller than the total number of tuples ($k \ll n$), making this overhead negligible while ensuring the sampling is not biased toward dominant, redundant patterns.

Stratified Sampling. The final phase extracts the minimal set of tuples required to satisfy the discovery recall guarantee. RepSampler iteratively extracts representative tuples from the managed buckets until the theoretical sampling lower bound N is reached, resulting in $O(N)$. While traditional discovery algorithms must process the full dataset n, RepSampler reduces the input size to a compact representative set N. Since $N \ll n$, RepSampler significantly lowers the computational cost of downstream mining while maintaining a rigorous recall guarantee.

D DETAILS OF CONFIDENCE CALIBRATION

We provide a formal justification for the adjusted confidence threshold δ'. Let P be the true probability that a CFD holds in relation r, and P_s be its empirical probability in sample r_s. According to the additive error bound ϵ defined in Theorem 2, we require:

$$P_r(|P_s - P| \leq \epsilon) \geq \delta$$

However, in practical sampling-based discovery, we can only observe the sample confidence δ'. To ensure the discovery is robust against the worst-case sampling drift, we consider the Relative Error Ratio $\alpha = \epsilon/\hat{\sigma}$. Following the concentration of measure for Bernoulli trials, the relationship between the observed confidence δ' and the target confidence δ is governed by the tails of the binomial distribution.

By applying the Linear Compensation Principle for the bias induced by ϵ [69], we observe that the effective confidence interval shrinks as ϵ increases. Specifically, the probability of a "true discovery" is attenuated by a factor proportional to the error margin. To maintain the reliability of the implication $X \rightarrow Y$, we solve for δ' such that:

$$\delta = \delta' \cdot \frac{1 + \frac{\epsilon}{4}}{1 - \frac{\epsilon}{4}}.$$

Rearranging this to isolate the required sample confidence, we obtain:

$$\delta' = \frac{1 - \frac{\epsilon}{4}}{1 + \frac{\epsilon}{4}} \delta.$$

This calibration ensures that δ' is sufficiently conservative to absorb the maximum expected variance $Var(X)$ within the sample, thereby guaranteeing that any CFD satisfying δ' in r_s possesses at least confidence δ in r. This proof decouples the adjustment from specific search strategies (like BFS [45]) and anchors it purely in Sampling Variance Compensation.

E ADDITIONAL EXPERIMENTAL DETAILS

E.1 Computation reduction of AttrFinder

The computation reduction in AttrFinder is the ratio of the lattice traversal computational overhead in the sub-tables to that of the original instance r, excluding representative sampling and pruning. It can be expressed as: $(\sum_{i=1}^{l} m_i \times d^{m_i})/(m \times d^m)$, where m is the number of attributes in r, d is the average domain size, l is the number of sub-tables, and m_i is the number of attributes in the i-th sub-table. In the experiment, AttrFinder identified 178, 236, 412, 104, 52, 214, and 152 correlated vector sets on the RT, Census, Crop, Flight, Adult, Syn-1, and Syn-2 datasets, respectively. Compared to direct mining, this leads to significant computational overhead reductions of at least $\frac{84}{d_{\text{RT}}^{45}}$, $\frac{63}{d_{\text{Census}}^{33}}$, $\frac{165}{d_{\text{Crop}}^{105}}$, $\frac{46}{d_{\text{Flight}}^{14}}$, $\frac{30}{d_{\text{Adult}}^{6}}$, $\frac{89}{d_{\text{Syn-1}}^{28}}$, and $\frac{78}{d_{\text{Syn-2}}^{18}}$ on these datasets (calculated based on the largest sub-table).

E.2 Computation reduction of RepSampler

The computation reduction in RepSampler is the ratio of the number of tuples in the sample (n_s) to the total number of tuples in the original relation (n), i.e., n_s/n. RepSampler selected 9,603, 12,447, 15,090, 13,483, 11,238, 9,928, and 9,274 representative tuples from the RT, Census, Crop, Flight, Adult, Syn-1, and Syn-2 datasets, respectively. This selection yielded computation reductions of 0.078, 0.062, 0.046, 0.013, 0.236, 0.496, and 0.464, respectively. Notably, the computational overhead exhibits a linear decrease as the sample size n_s is reduced.

Attack Patterns	Count	Normal Patterns	Count
DOS SYN Hping	94,659	Amazon Alexa	86,842
ARP poisioning	7,750	MQTT	8,108
NMAP UDP SCAN	2,590	Thing speak	4,146
NMAP XMAS TREE SCAN	2,010	Wipro bulb	253
NMAP OS DETECTION	2,000		
NMAP TCP scan	1,002		
DDOS Slowloris	534		
Metasploit Brute Force SSH	37		
NMAP FIN SCAN	28		

Table 7: Distribution of Attack and Normal patterns in the RT-IOT dataset.

E.3 Details of Experimental datasets

We provide a comprehensive overview of the five real world datasets used in our experiments, focusing on their structural characteristics and relevance to CFD discovery.

(1) RT-IOT (RT). The RT-IOT-2022 dataset is a comprehensive resource derived from a real time IOT infrastructure [65]. Unlike traditional static datasets, it integrates a diverse range of physical IOT devices including ThingSpeak LED, Wipro Bulb, and MQTT Temp alongside sophisticated network attack methodologies. The dataset is designed to provide a general representation of real world scenarios by capturing both normal operations and adversarial behaviors. To ensure high fidelity in network traffic analysis, the bidirectional attributes of the traffic were meticulously captured using the Zeek network monitoring tool and the Flowmeter plugin. This process resulted in a rich feature set that allows researchers to analyze complex traffic patterns across various protocols.

The dataset contains a total of 123,112 instances, categorized into Attack patterns and Normal patterns as shown in Table 7. In our research, this dataset serves as a benchmark for mining CFDs. Specifically, the coexistence of normal and malicious traffic allows us to identify dependencies that hold under normal conditions but are violated during specific attack phases. For example, a CFD may capture the stable relationship between a device type and its expected traffic volume, which becomes an essential constraint for identifying anomalous behavior in IOT environments.

(2) Census-Income (Census). The CensusIncome dataset consists of weighted census records extracted from the 1994 and 1995 Current Population Surveys conducted by the U.S. Census Bureau. This dataset is a high dimensional resource containing 41 demographic and employment related variables for each individual, including features such as age, class of worker, education, and marital status. It provides a total of 199,523 training instances and 99,762 test instances, which were split in approximately a two thirds to one third proportion. A critical attribute of this dataset is the instance weight (MARSUPWT), which indicates the number of people in the actual population that each record represents due to stratified sampling. While this field is essential for deriving statistical conclusions about the general population, it is typically excluded from the feature set used in classification models. The primary task associated with this dataset is a binary classification problem. Similar to the original Adult database, incomes have been binned at the 50,000 dollar level. However, the target variable in this specific version was derived from the total person income field (PTOTVAL) rather than adjusted gross income, which introduces distinct statistical characteristics.

For the purpose of mining CFDs, the Census Income dataset is particularly valuable due to its rich set of categorical and numerical attributes. It allows for the discovery of complex dependencies between demographic profiles and socio economic status, such as the relationship between occupation codes and wage per hour conditioned on specific industry sectors.

(3) Crop mapping using fused optical radar data (Crop). The Crop dataset is a fused bi temporal resource designed for sophisticated cropland classification near Winnipeg, Manitoba, Canada. Collected in 2012, this dataset combines optical imagery from RapidEye satellites with Polarimetric Synthetic Aperture Radar (PolSAR) data from the Unmanned Aerial Vehicle Synthetic Aperture Radar (UAVSAR) system. By integrating these complementary sensing modalities, the dataset captures a significant number of temporal, spectral, textural, and polarimetric features that are essential for distinguishing between various crop types.

The dataset contains 175 attributes in total, structured around two specific observation dates: July 05 and July 14, 2012. The feature space is divided into several categories: (1) Polarimetric Features: 49 radar features per date (f1 to f98) capturing scattering properties such as sigHH, sigVV, and complex decompositions like Cloude Pottier parameters (Entropy, Anisotropy) and Freeman Durden components. (2) Optical Features: 38 optical features per date (f99 to f174) including spectral bands (Blue, Green, Red, Red edge, NIR) and derived vegetation indices such as NDVI, EVI, and SAVI, alongside textural metrics derived from Principal Component Analysis. There are seven distinct crop type classes identified in this dataset: Corn, Peas, Canola, Soybeans, Oats, Wheat, and Broadleaf.

In our study, the Crop dataset serves as a benchmark for mining CFDs across multi modal sensor data. The rich set of 175 attributes allows for the discovery of dependencies where spectral signatures from optical sensors are constrained by polarimetric backscatter measurements. For instance, a CFD might define a stable relationship between a specific crop type and its expected NDVI range, conditioned on the stability of its radar backscatter cross section during different growth stages.

(4) 2015 Flight Delays and Cancellations (Flight). The Flight dataset, specifically the 2015 Flight Delays and Cancellations resource, provides an exhaustive record of domestic flight operations within the United States. It encompasses nearly 5.8 million flights, capturing critical metrics such as scheduled and actual departure times, arrival delays, and cancellation reasons. The dataset is structured to facilitate multi dimensional analysis across three primary entities: Airlines (14 major carriers), Airports (322 locations), and Time (year, month, day, and day of week). The raw data underwent a rigorous cleaning process to ensure analytical integrity. This included converting blank entries into NA values, omitting records with missing scheduled time or air time, and merging flight records with airport and airline lookup tables to resolve numeric codes into descriptive names. Statistical profiling reveals that approximately 1.6 percent of all flights were cancelled, with over half of these cancellations attributed to weather conditions (Category B), followed by airline related issues (Category A) and air system delays (Category C).

For the discovery of CFDs, this dataset is particularly valuable due to the intricate relationships between categorical attributes (e.g.,

Origin Airport, Airline) and numerical performance metrics (e.g., Departure Delay, Taxi In time). It allows for the identification of dependencies where the probability of a delay is constrained by specific temporal or geographical contexts. For example, a CFD might capture a stable constraint where flights operated by a specific carrier from a certain hub consistently maintain a lower taxi out duration, provided the weather conditions remain within normal parameters.

(5) Adult. The Adult dataset, also widely recognized as the Census Income dataset, was extracted by Barry Becker from the 1994 United States Census database. To ensure data quality, a set of reasonably clean records was filtered based on specific demographic and economic conditions, including individuals older than 16 years, an adjusted gross income exceeding 100 dollars, and a recorded work volume of more than zero hours per week.

The primary prediction task for this dataset is a binary classification problem: determining whether an individual's annual income exceeds the 50,000 dollar threshold. The dataset comprises 14 diverse attributes that capture a holistic view of an individual's socio economic profile: (1) Categorical Attributes: These include workclass (e.g., Private, Federal gov), education level, marital status, occupation, relationship role, race, sex, and native country. (2) Continuous Attributes: These encompass age, final weight (fnlwgt), education num, capital gain, capital loss, and hours per week.

In the context of our research on CFDs, the Adult dataset provides a robust environment for discovering complex logical constraints. Its mix of categorical and continuous variables allows for the identification of dependencies where professional attributes (such as occupation and education) are functionally linked to income levels under specific demographic conditions (such as workclass or age groups). For instance, a CFD might define a high confidence dependency between a doctorate degree and the high income class, specifically within the context of private sector employment.